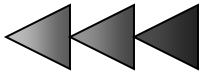
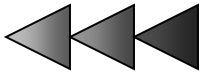
Department of Electrical Engineering, National Taiwan University of Science and Technology



Chapter 2

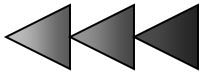
Design for Testability

Design For Testability-Contents



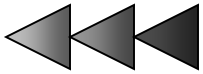
- Introduction
- Testability Analysis
- Design for Testability Basics
- Scan Cells Designs
- Scan Architectures
- Scan Design Rules
- Scan Design Flow
- Special-Purpose Scan Designs
- RTL Design for Testability
- Concluding Remarks

Introduction



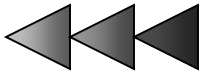
- History
  - During early years, design and test were separate
    - The final quality of the test was determined by keeping track of the number of defective parts shipped to the customer
    - Defective parts per million (DPM) shipped was a final test score.
    - This approach worked well for small-scale integrated circuit
  - During 1980s, fault simulation was used
    - Failed to improve the circuit's fault coverage beyond 80%
  - Increased test cost and decreased test quality lead to DFT engineering

Introduction (Cont.)



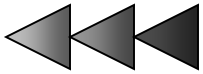
- History
  - Various testability measures & *ad hoc* testability enhancement methods
    - To improve the testability of a design
    - To ease sequential ATPG (automatic test pattern generation)
    - Still quite difficult to reach more than 90% fault coverage
  - Structured DFT
    - To conquer the difficulties in controlling and observing the internal states of sequential circuits
    - Scan design is the most popular structured DFT approach
  - Design for testability (DFT) has migration recently
    - From gate level to register-transfer level (RTL)

Testability Analysis



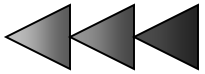
- Testability:
  - A relative measure of the effort or cost of testing a logic circuit
- Testability Analysis:
  - The process of assessing the testability of a logic circuit
- Testability Analysis Techniques:
  - Topology-based Testability Analysis
    - SCOAP-*Sandia Controllability/Observability Analysis Program*
  - *Probability-based testability analysis*
  - Simulation-based Testability Analysis

Testability Analysis–*SCOAP*



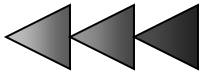
- *Controllability*
  - Reflects the difficulty of setting a signal line to a required logic value from primary inputs
- *Observability*
  - Reflects the difficulty of propagating the logic value of the signal line to primary outputs

Testability Analysis–*SCOAP*



- Calculates six numerical values for each signal *s* in a logic circuit
  - CC0(*s*): combinational 0-controllability of *s*
  - CC1(*s*): combinational 1-controllability of *s*
  - CO(*s*): combinational observability of *s*
  - SC0(*s*): sequential 0-controllability of *s*
  - SC1(*s*): sequential 1-controllability of *s*
  - SO(*s*): sequential observability of *s*

Testability Analysis – ***SCOAP***



- The value of controllability measures range between 1 to infinite
- The value of observability measures range between 0 to infinite
  - The CC0 and CC1 values of a primary input are set to 1
  - The SC0 and SC1 values of a primary input are set to 0
  - The CO and SO values of a primary output are set to 0

SCOAP-Combinational Controllability Calculation Rules

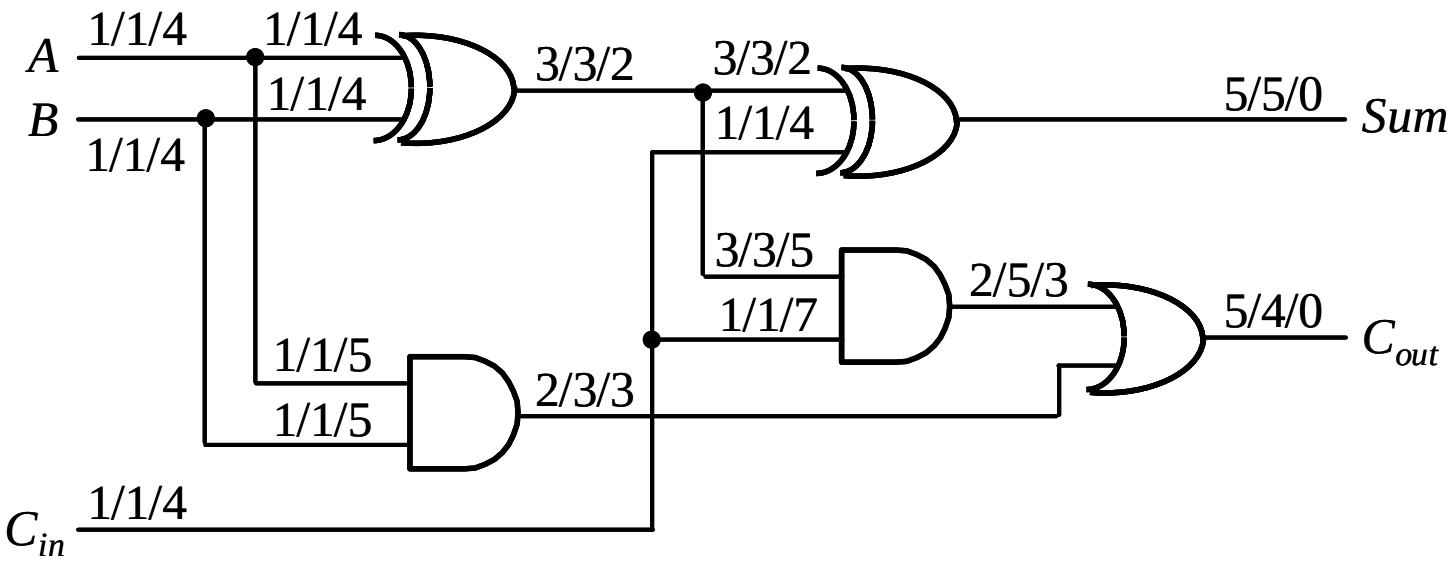
	0-controllability (Primary input, output, branch)	**1-controllability** (Primary input, output, branch)
Primary Input	1	1
AND	*min* {input 0-controllabilities} + 1	Σ (input 1-controllabilities) + 1
OR	Σ (input 0-controllabilities) + 1	*min* {input 1-controllabilities} + 1
NOT	Input 1-controllability + 1	Input 0-controllability + 1
NAND	Σ (input 1-controllabilities) + 1	*min* {input 0-controllabilities} + 1
NOR	*min* {input 1-controllabilities} + 1	Σ (input 0-controllabilities) + 1
BUFFER	Input 0-controllability + 1	Input 1-controllability + 1
XOR	*min* {CC1(*a*)+CC1(*b*), CC0(*a*)+CC0(*b*)} + 1	*min* {CC1(*a*)+CC0(*b*), CC0(*a*)+CC1(*b*)} + 1
XNOR	*min* {CC1(*a*)+CC0(*b*), CC0(*a*)+CC1(*b*)} + 1	*min* {CC1(*a*)+CC1(*b*), CC0(*a*)+CC0(*b*)} + 1
Branch	Stem 0-controllability	Stem 1-controllability

SCOAP-Combinational Controllability/ Observability Rules

	Observability (Primary output, input, stem)
Primary Output	0
AND/NAND	Σ (output observability, 1-controllabilities of other inputs) + 1
OR/NOR	Σ (output observability, 0-controllabilities of other inputs) + 1
NOT/BUFFER	Output observability + 1
XOR/XNOR	*a*: Σ (output observability, *min* {CC0(*b*), CC1(*b*)}) + 1 *b*: Σ (output observability, *min* {CC0(*a*), CC1(*a*)}) + 1
Stem	*min* {branch observabilities}

a, *b*: inputs of an XOR or XNOR gate

Testability Analysis–*SCOAP*

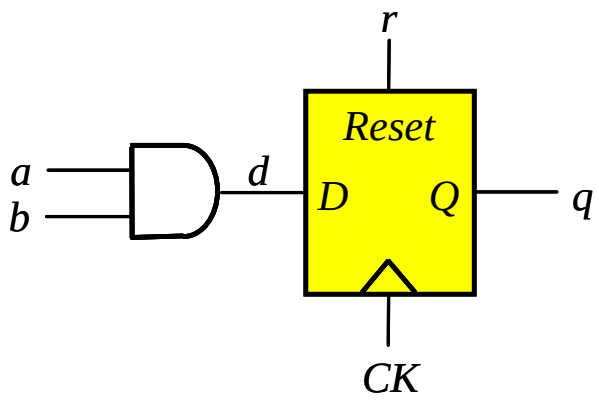


Example of Combinational SCOAP measures

*v*1/*v*2/*v*3 represents the signal's 0-controllability (*v*1), 1-controllability (*v*2), and observability (*v*3)

Testability Analysis-*SCOAP*

Sequential Controllability and Observability Calculation



The combinational and sequential controllability measures of signal *d:*

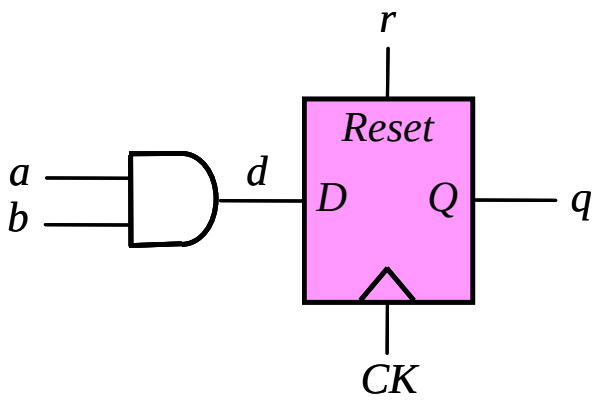
$$CC0(d) = \min\{CC0(a), CC0(b)\} + 1$$
$$SC0(d) = \min\{SC0(a), SC0(b)\}$$
$$CC1(d) = CC1(a) + CC1(b) + 1$$
$$SC1(d) = SC1(a) + SC1(b)$$

Testability Analysis-*SCOAP*

The combinational and sequential controllability and observability measures of *q:*

$$CC0(q) = \min\{CC0(d) + CC0(CK) + CC1(CK) + CC0(r), CC1(r) + CC0(CK)\}$$
$$SC0(q) = \min\{SC0(d) + SC0(CK) + SC1(CK) + SC0(r) + 1, SC1(r) + SC0(CK)\}$$

$$CC1(q) = CC1(d) + CC0(CK) + CC1(CK) + CC0(r)$$
$$SC1(q) = SC1(d) + SC0(CK) + SC1(CK) + SC0(r) + 1$$



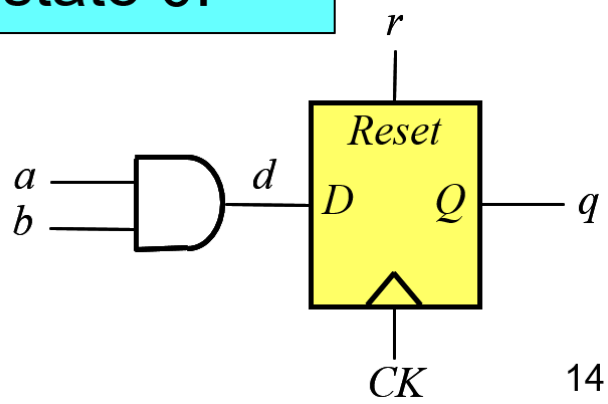
Testability Analysis-*SCOAP*

The data input *d* can be observed at *q* by holding the reset signal *r* at 0 and applying a rising clock edge to *CK:*

$$CO(d) = CO(q) + CC0(CK) + CC1(CK) + CC0(r)$$
$$SO(d) = SO(q) + SC0(CK) + SC1(CK) + SC0(r) + 1$$

Signal *r* can be observed by first setting *q* to 1, and then holding *CK* at the inactive state 0:

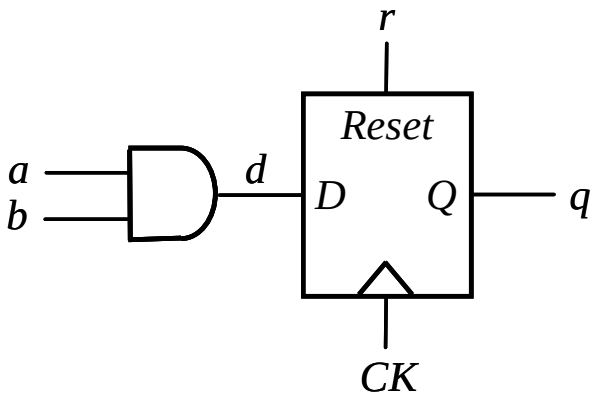
$$CO(r) = CO(q) + CC1(q) + CC0(CK)$$
$$SO(r) = SO(q) + SC1(q) + SC0(CK)$$



Testability Analysis-*SCOAP*

- Two ways to indirectly observe the clock signal *CK* at *q*:
  - set *q* to 1, *r* to 0, *d* to 0, and apply a rising clock edge at *CK*
  - set both *q* and *r* to 0, *d* to 1, and apply a rising clock edge at *CK*

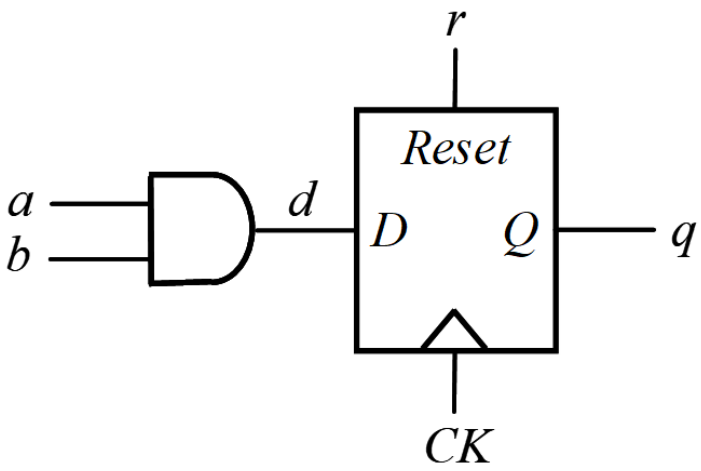
$$CO(CK) = CO(q) + CC0(CK) + CC1(CK) + CC0(r) + \min\{CC0(d) + CC1(q),\ CC1(d) + CC0(q)\}$$
$$SO(CK) = SO(q) + SC0(CK) + SC1(CK) + SC0(r) + \min\{SC0(d) + SC1(q),\ SC1(d) + SC0(q)\} + 1$$



Testability Analysis-*SCOAP*

The combinational and sequential observability measures for both inputs *a* and *b:*

$$CO(a) = CO(d) + CC1(b) + 1$$
$$SO(a) = SO(d) + SC1(b)$$
$$CO(b) = CO(d) + CC1(a) + 1$$
$$SO(b) = SO(d) + SC1(a)$$



Probability-Based Testability Analysis

- Used to analyze the random testability of the circuit
  - C0(*s*): probability-based 0-controllability of *s*
  - C1(*s*): probability-based 1-controllability of *s*
  - O(*s*): probability-based observability of *s*
- Range between 0 and 1
- C0(*s*) + C1(*s*) = 1

Probability-based controllability calculation rules

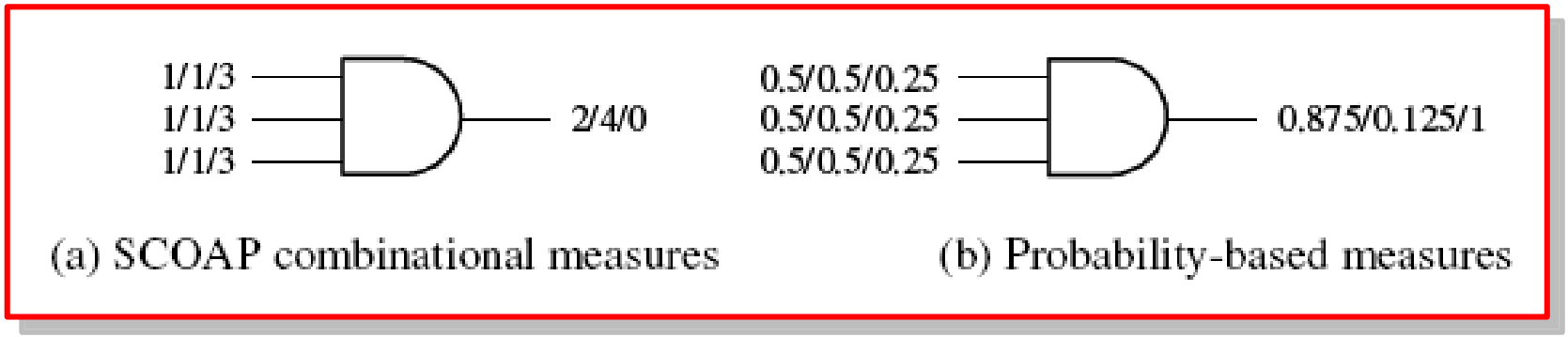
	0-controllability (Primary input, output, branch)	**1-controllability** (Primary input, output, branch)
Primary Input	p_0	$p_1 = 1 - p_0$
AND	1 – (output 1-controllability)	Π (input 1-controllabilities)
OR	Π (input 0-controllabilities)	1 – (output 0-controllability)
NOT	Input 1-controllability	Input 0-controllability
NAND	Π (input 1-controllabilities)	1 – (output 0-controllability)
NOR	1 – (output 1-controllability)	Π (input 0-controllabilities)
BUFFER	Input 0-controllability	Input 1-controllability
XOR	1 – 1-controllabilty	Σ (C1(a) × C0(b), C0(a) × C1(b))
XNOR	1 – 1-controllability	Σ (C0(a) × C0(b), C1(a) × C1(b))
Branch	Stem 0-controllability	Stem 1-controllability

Probability-based observability calculation rules

	Observability (Primary output, input, stem)
Primary Output	1
AND / NAND	Π (output observability, 1-controllabilities of other inputs)
OR / NOR	Π (output observability, 0-controllabilities of other inputs)
NOT / BUFFER	Output observability
XOR / XNOR	*a*: Π (output observability, *max* {0-controllability of *b*, 1-controllability of *b*}) *b*: Π (output observability, *max* {0-controllability of *a*, 1-controllability of *a*})
Stem	*max* {branch observabilities}

a, *b*: inputs of an XOR or XNOR gate

Difference between SCOAP testability measures and probability-based testability measures

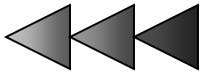


*v*1/*v*2/*v*3 represents the signal's 0-controllability (*v*1), 1-controllability (*v*2), and observability (*v*3)

Design for Testability Basics

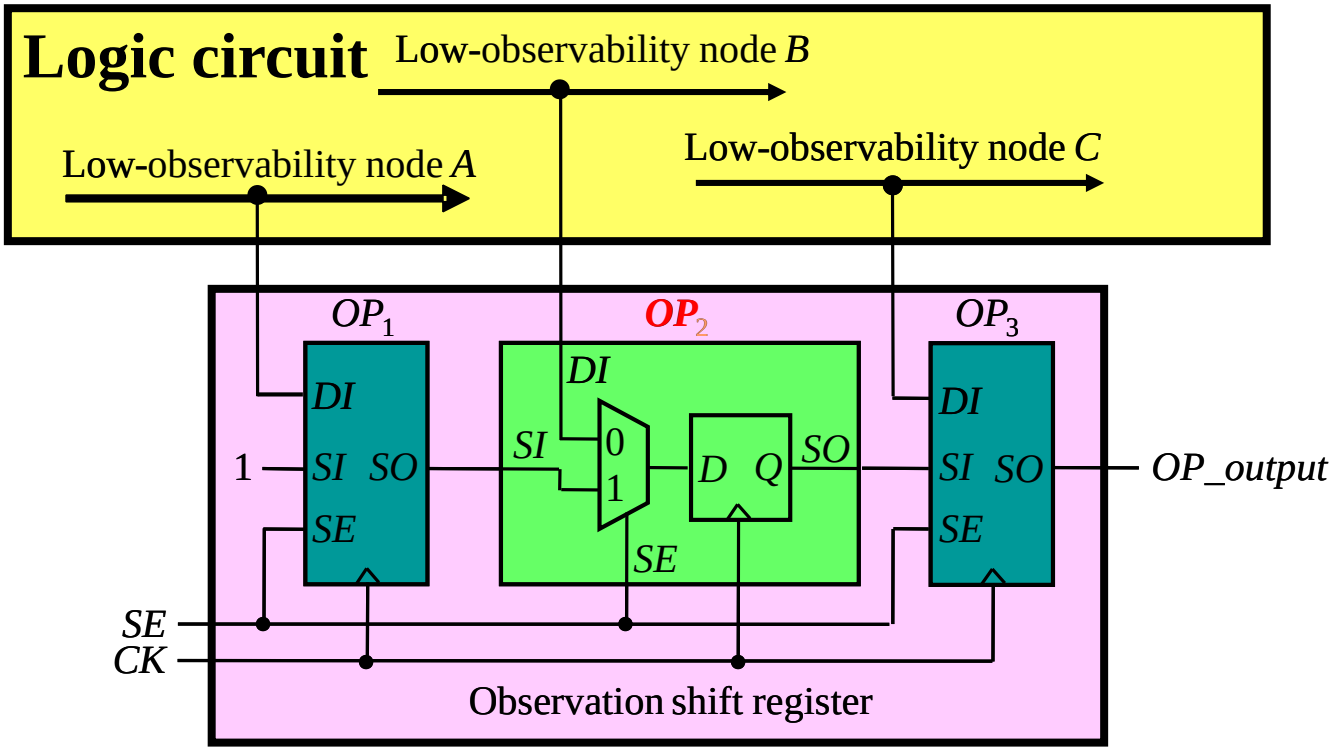
- *Ad hoc* DFT
  - Effects are local and not systematic
  - Not methodical
  - Difficult to predict
- A structured DFT
  - Easily incorporated and budgeted
  - Yield the desired results
  - Easy to automate

Ad Hoc *Approach*



- Typical *ad hoc* DFT techniques
  - Insert test points
  - Avoid asynchronous set/reset for storage elements
  - Avoid combinational feedback loops
  - Avoid redundant logic
  - Avoid asynchronous logic
  - Partition a large circuit into small blocks

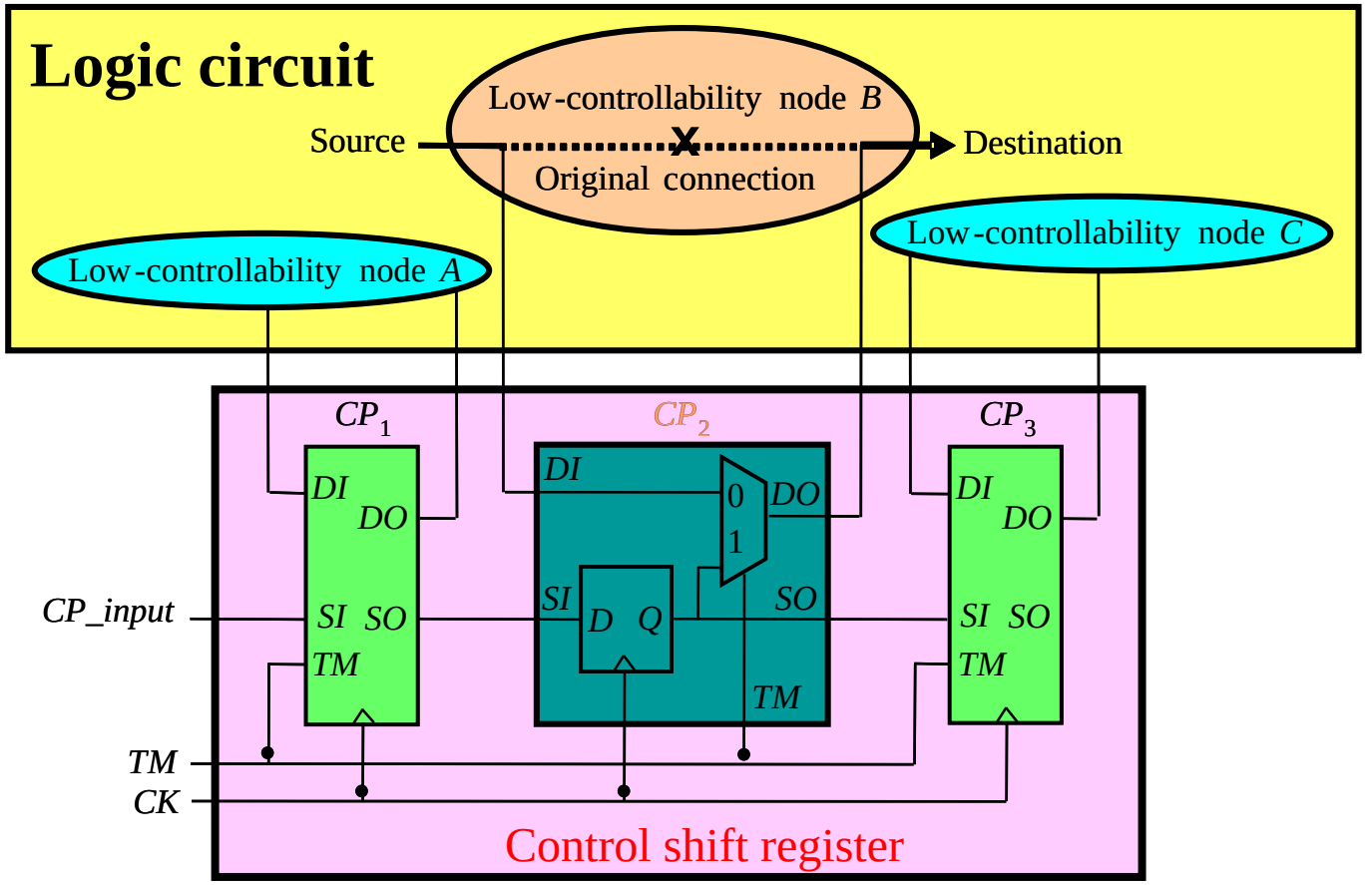
Ad Hoc *Approach–Test Point Insertion*



Observation point insertion

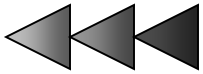
OP2 shows the structure of an observation register, which is composed of a multiplexer (MUX) and a D flip-flop.

Ad Hoc Approach–Test Point Insertion



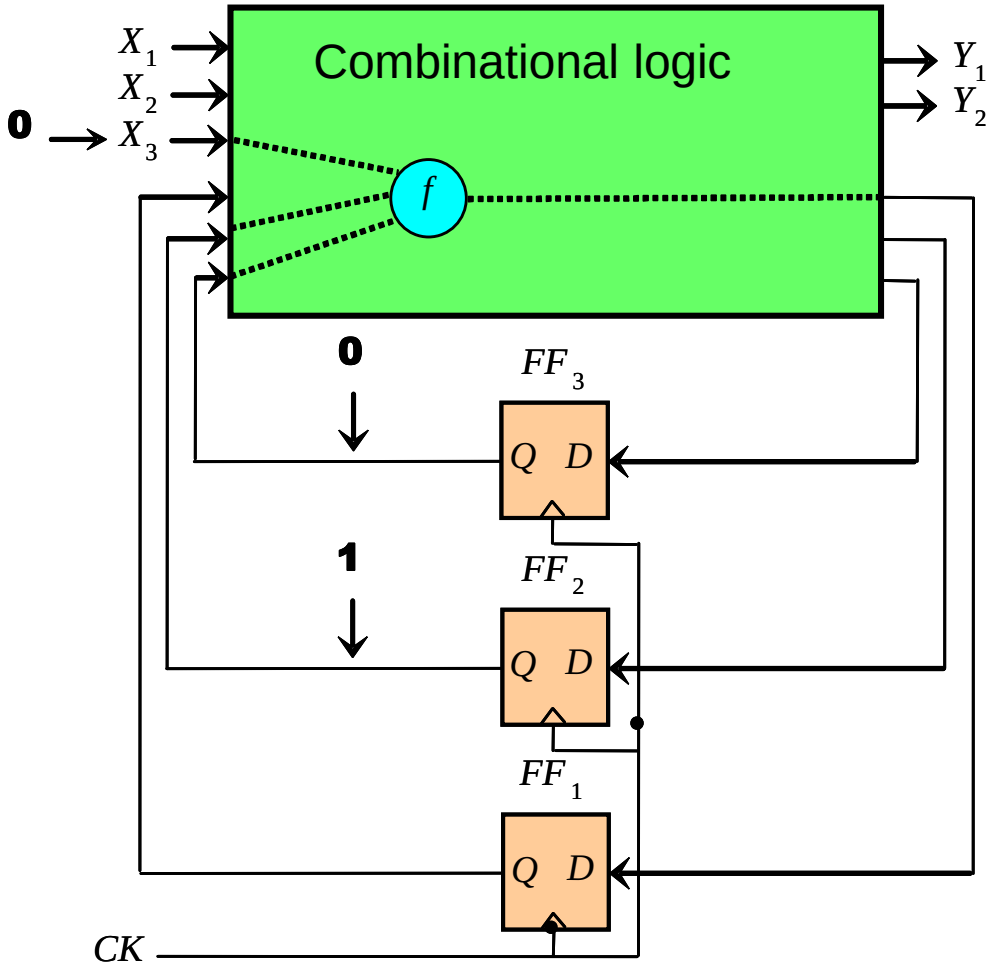
Control point insertion

Structured Approach

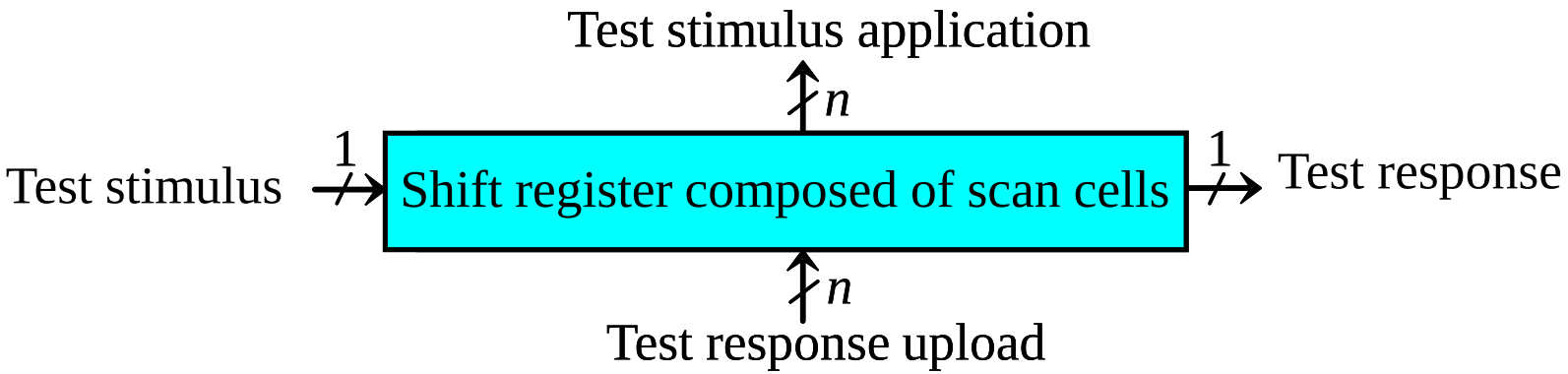


- Scan design
  - Convert the sequential design into a scan design
  - Three modes of operation
    - Normal mode
      - All test signals are turned off
      - The scan design operates in the original functional configuration
    - Shift mode
    - Capture mode
      - In both shift and capture modes, a test mode signal *TM* is often used to turn on all test-related fixes

Structured Approach-Scan Design

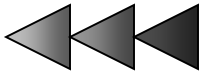


Structured Approach-Scan Design



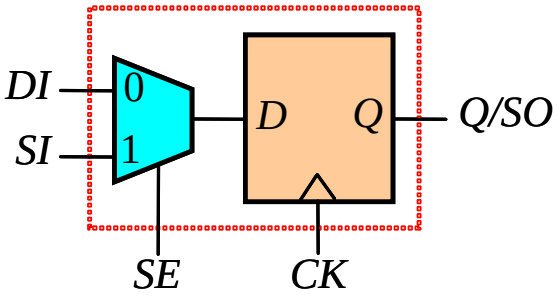
1. Converting selected storage elements in the design into scan cells.
2. Stitching them together to form scan chains.

Scan Cell Design



- A scan cell has two inputs: data input and scan input
  - In normal/capture mode, data input is selected to update the output
  - In shift mode, scan input is selected to update the output
- Three widely used scan cell designs
  - Muxed-D Scan Cell
  - Clocked-Scan Cell
  - LSSD Scan Cell

Muxed-D Scan Cell

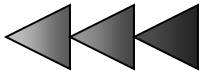


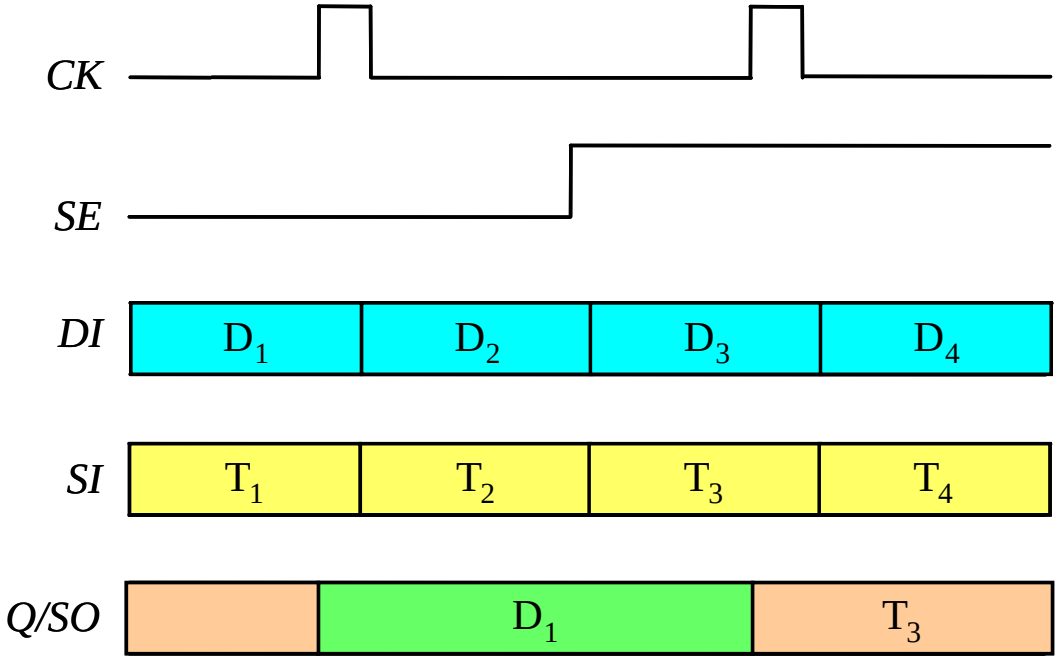
Edge-triggered muxed-D scan cell

This scan cell is composed of a D flip-flop and a multiplexer.

The multiplexer uses an additional scan enable input *SE* to select between the data input *DI* and the scan input *SI*.

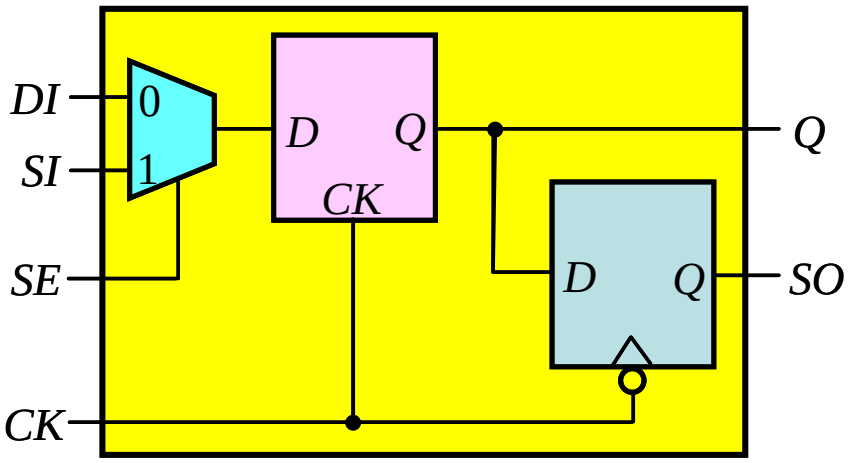
Muxed-D Scan Cell





Edge-triggered muxed-D scan cell design and operation

Muxed-D Scan Cell

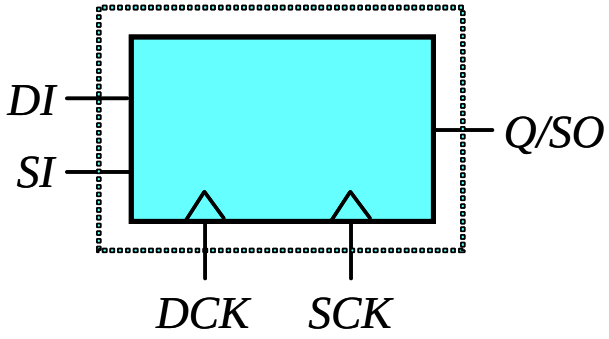


Level-sensitive/edge-triggered muxed-D scan cell design

This scan cell is composed of a multiplexer, a D latch, and a D flip-flop.

In this case, shift operation is conducted in an edge-triggered manner, while normal operation and capture operation is conducted in a level-sensitive manner.

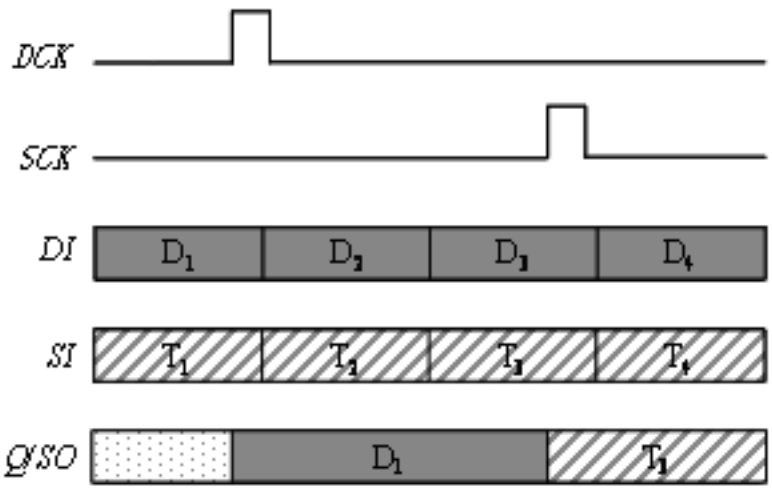
Clocked-Scan Cell



Clocked-scan cell

In the clocked-scan cell, input selection is conducted using two independent clocks, *DCK* and *SCK*.

Clocked-Scan Cell

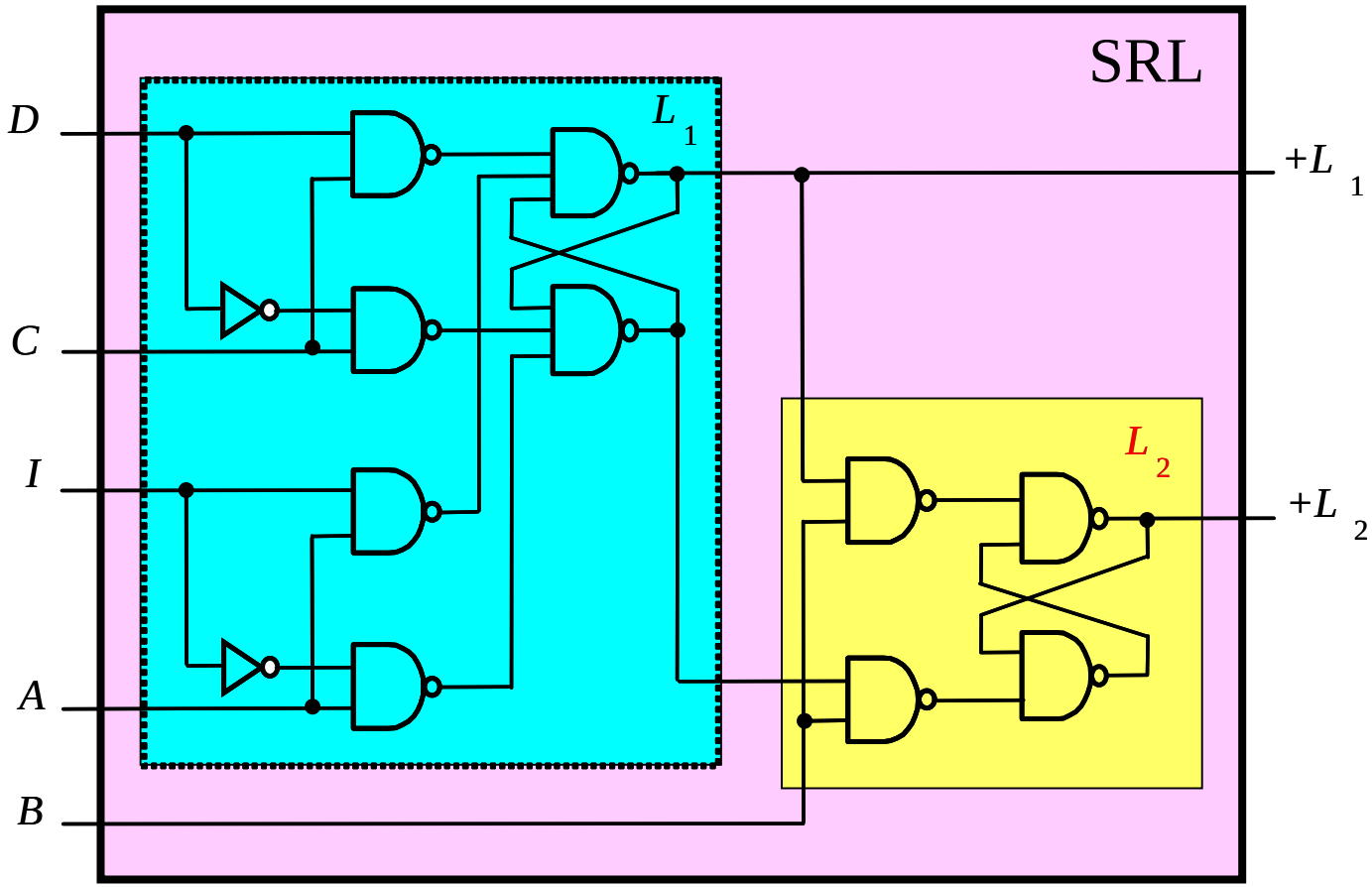


Clocked-scan cell design and operation

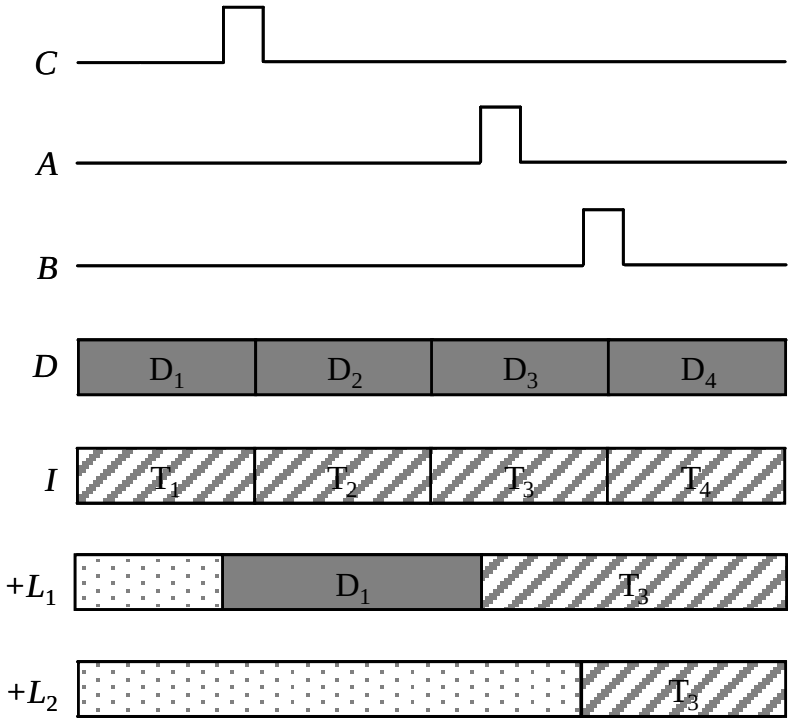
In normal/capture mode, the data clock *DCK* is used to capture the contents present at the data input *DI* into the clocked-scan cell.

In shift mode, the shift clock *SCK* is used to shift in new data from the scan input *SI* into the clocked -scan cell, while the content of the clocked-scan cell is being shifted out.

LSSD Scan Cell



LSSD Scan Cell

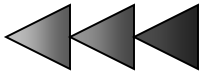


Polarity-hold SRL design and operation

In order to guarantee race-free operation, clocks *A*, *B*, and *C* are applied in a non-overlapping manner.

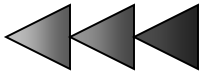
The master latch L_1 uses the system clock *C* to latch system data from the data input *D* and to output this data onto $+L_1$. Clock *B* is used after clock *A* to latch the system data from latch L_1 and to output this data onto $+L_2$.

Scan Architectures



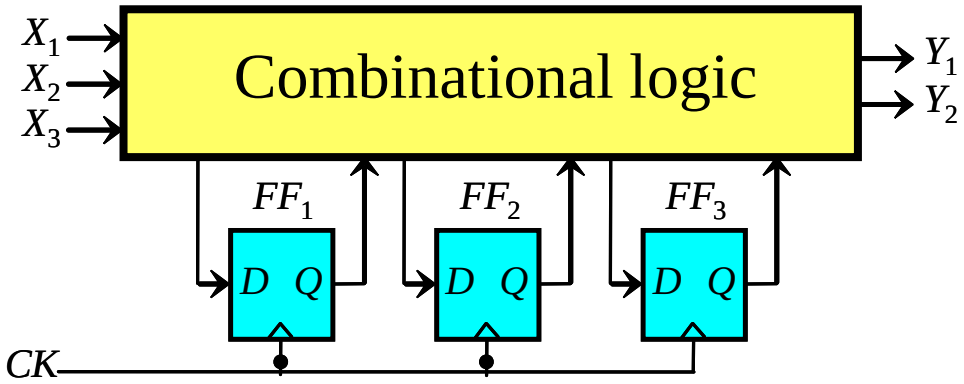
- Full-Scan Design
  - All or almost all storage element are converted into scan cells and combinational ATPG is used for test generation
- Partial-Scan Design
  - A subset of storage elements are converted into scan cells and sequential ATPG is typically used for test generation
- Random-Access Scan Design
  - A random addressing mechanism, instead of serial scan chains, is used to provide direct access to read or write any scan cell

Full-Scan Design



- All storage elements are replaced with scan cells
  - All inputs can be controlled
  - All outputs can be observed
- Advantage:
  - Converts sequential ATPG into combinational ATPG
- **Almost full-scan design**
  - A small percentage of storage elements are not replaced with scan cells
    - For performance reasons
      - Storage elements that lie on critical paths
    - For functional reasons
      - Storage elements driven by a small clock domain that are deemed too insignificant to be worth the additional scan insertion effort

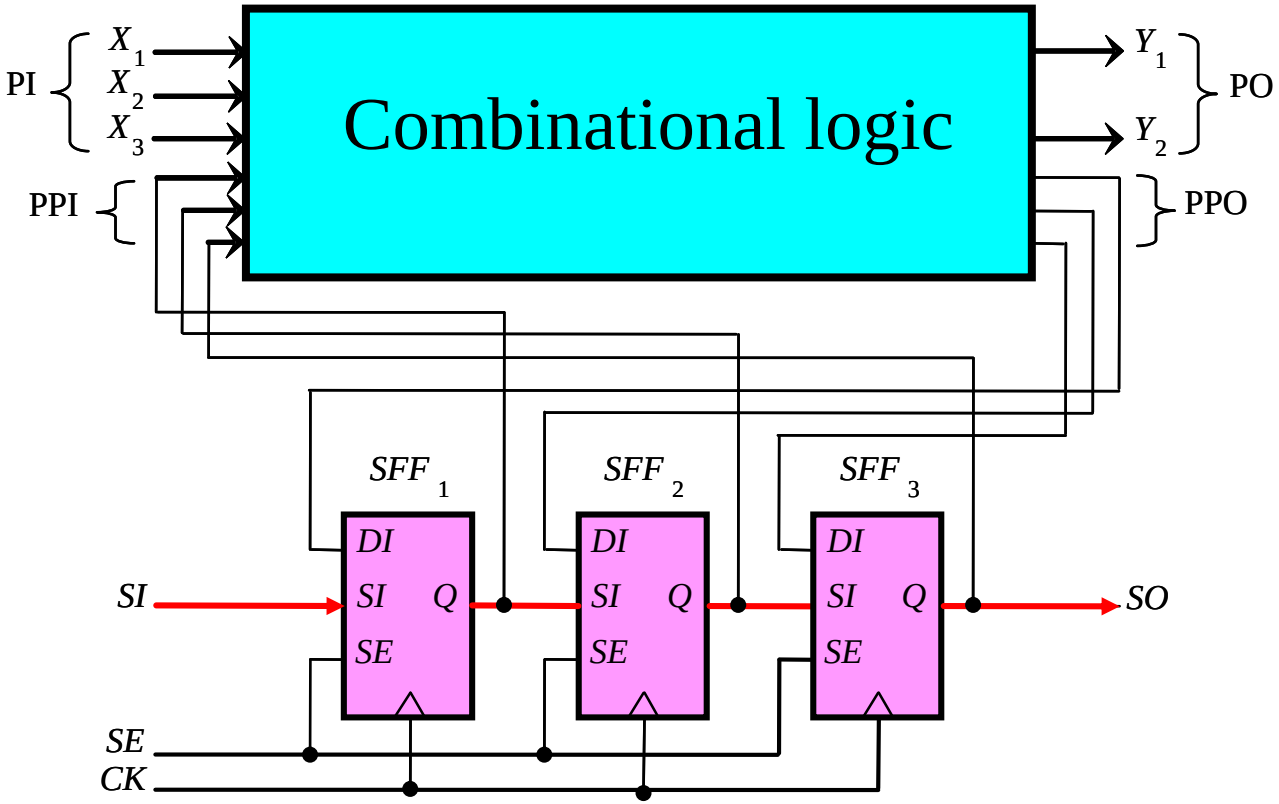
Muxed-D Full-Scan Design



Sequential circuit example

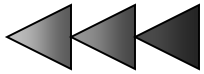
The three D flip-flops, FF_1, FF_2 and FF_3, are replaced with three muxed-D scan cells, SFF_1, SFF_2 and SFF_3, respectively.

Muxed-D Full-Scan Design



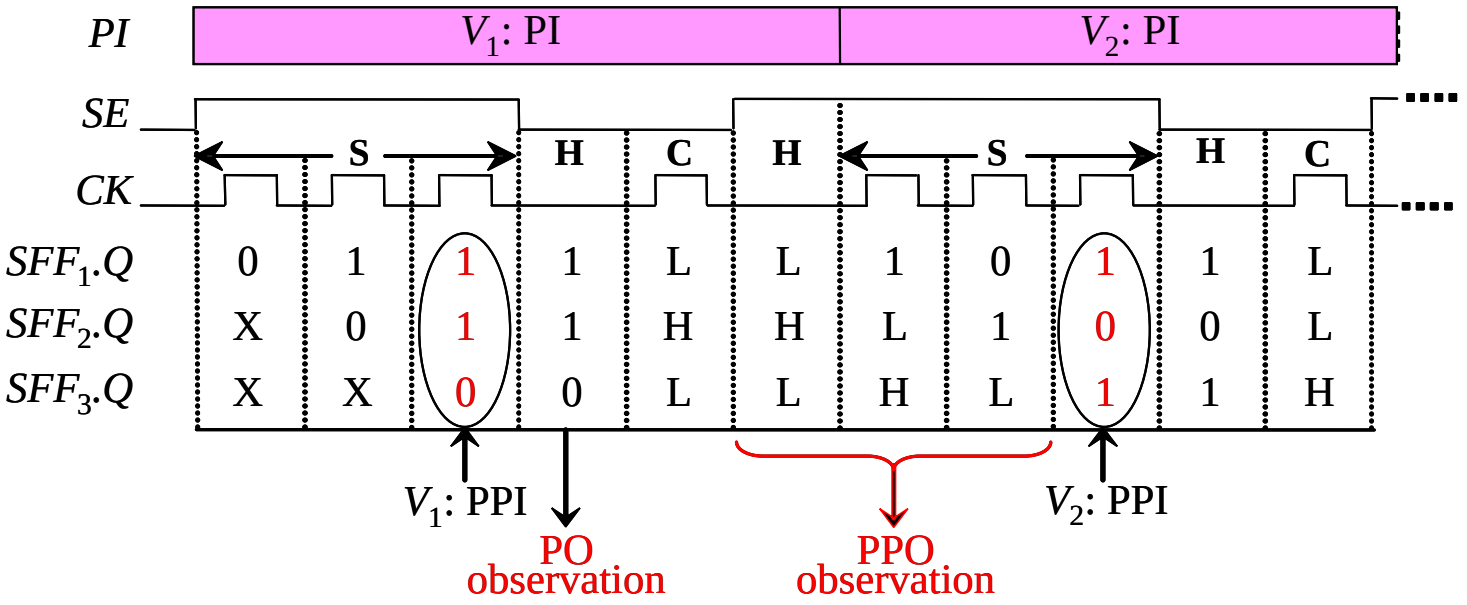
Muxed-D full-scan circuit

Muxed-D Full-Scan Design



- *Primary inputs* (PIs)
  - the external inputs to the circuit
  - can be set to any required logic values
  - set directly in parallel from the external inputs
- *Pseudo primary inputs* (PPIs)
  - the scan cell outputs
  - can be set to any required logic values
  - are set serially through scan chain inputs
- *Primary outputs* (POs)
  - the external outputs of the circuit
  - can be observed
  - are observed directly in
  - parallel from the external outputs
- *Pseudo primary outputs* (PPOs)
  - the scan cell inputs
  - can be observed
  - are observed serially through scan chain outputs

Muxed-D Full-Scan Design



S: shift operation / **C**: capture operation / **H**: hold cycle

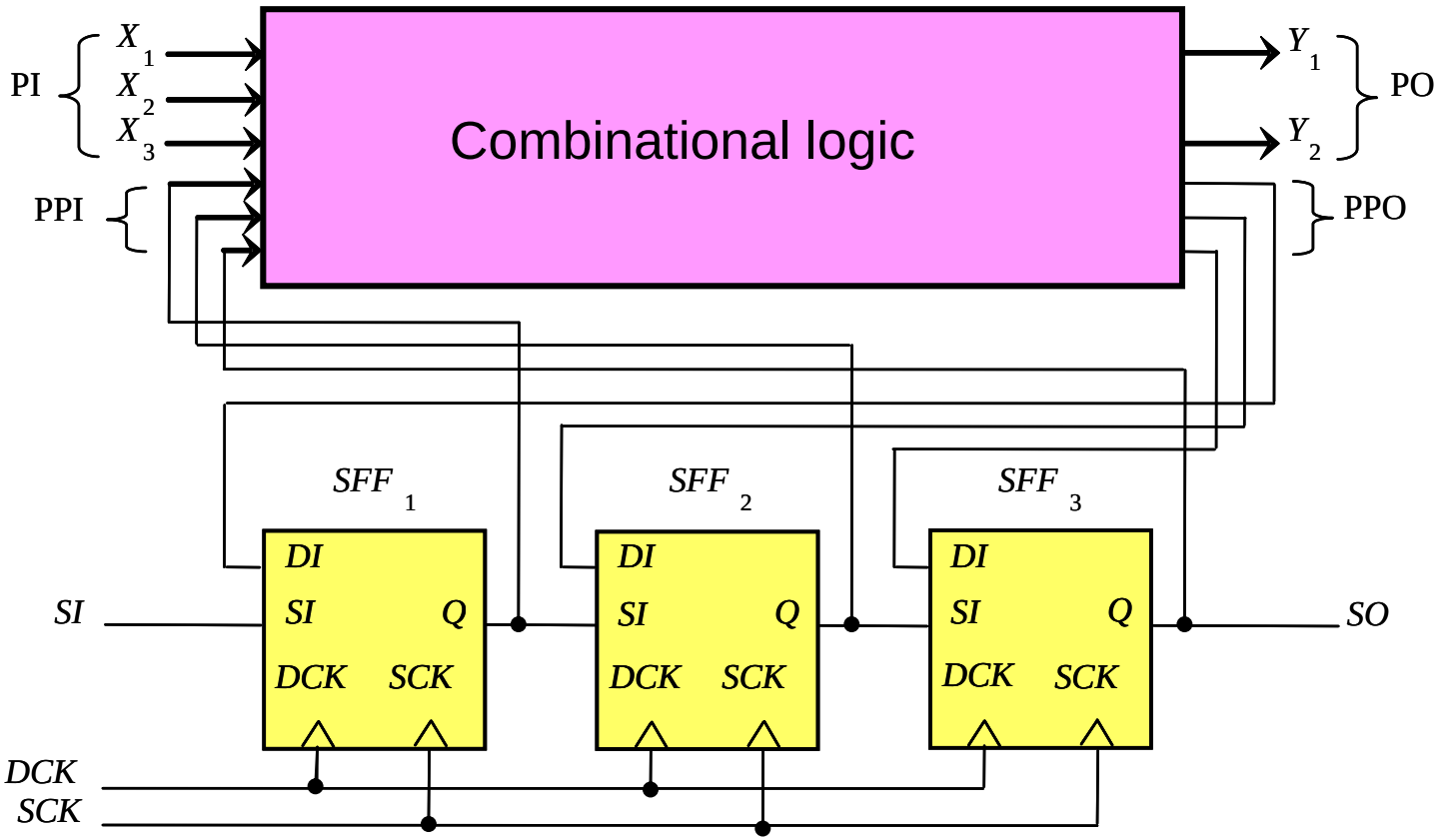
Test operations

Muxed-D Full-Scan Design

Circuit Operation type	Scan cell mode	*TM*	SE
Normal	Normal	0	0
Shift Operation	Shift	1	1
Capture Operation	Capture	1	0

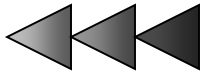
Circuit operation type and scan cell mode

Clocked Full-Scan Design



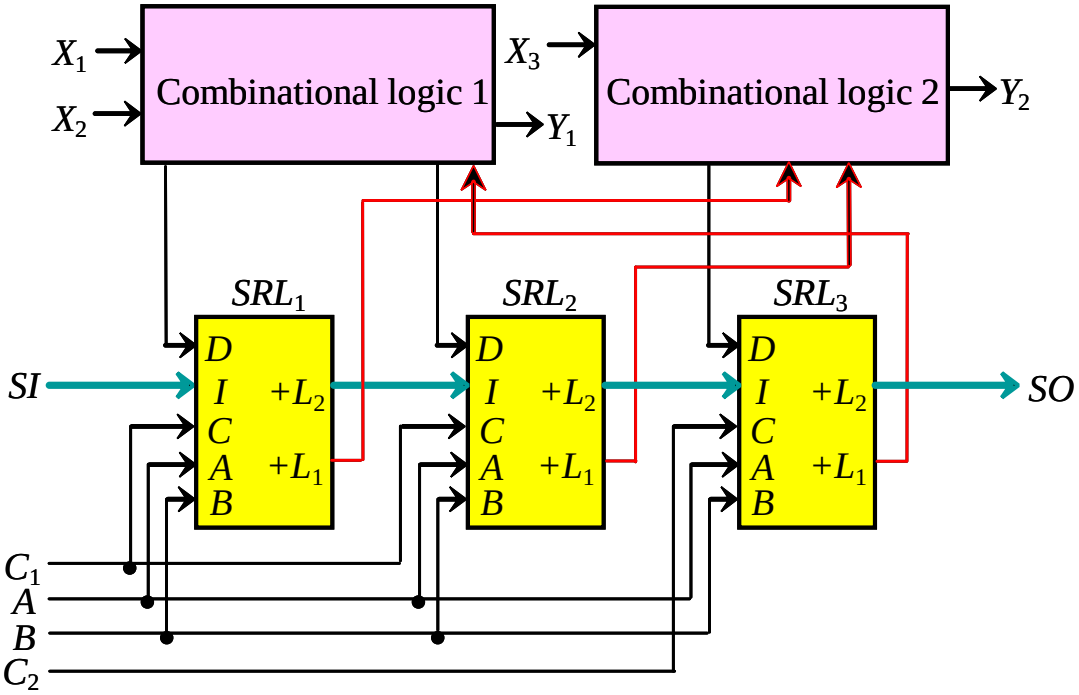
Clocked full-scan circuit

LSSD Full-Scan Design



- Single-latch design
- Double-latch design

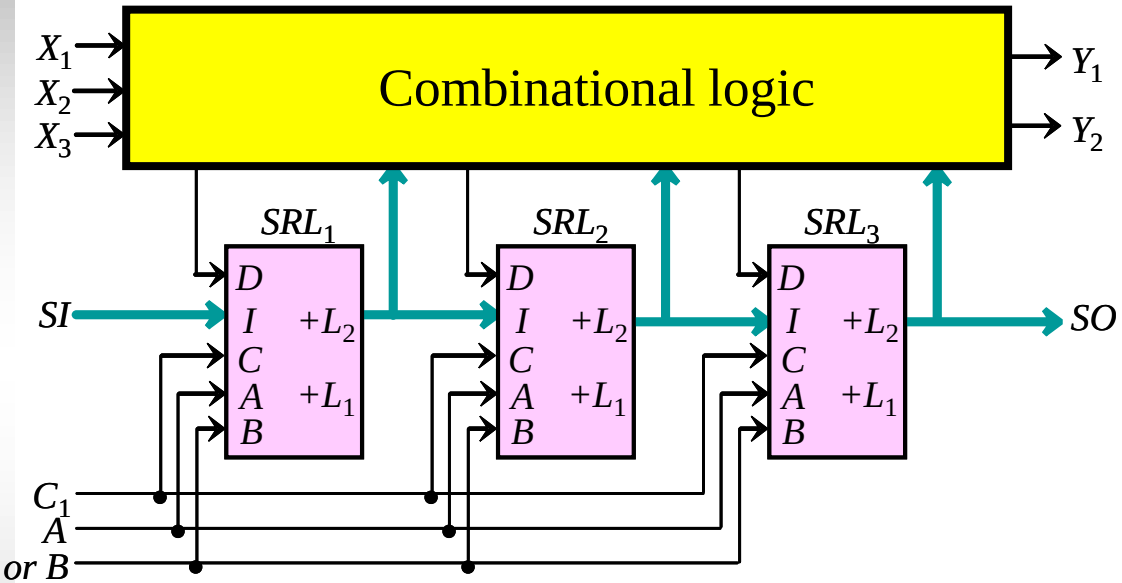
LSSD Full-Scan Design



Single-latch design

The output port $+L_1$ of the master latch L_1 is used to drive the combinational logic of the design. In this case, the slave latch L_2 is only used for scan testing.

LSSD Full-Scan Design



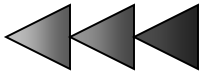
Double-latch design

In normal mode, the C_1 and C_2 clocks are used in a non-overlapping Manner.

During the shift operation, clocks *A* and *B* are applied in a non-overlapping manner, the scan cells $SRL_1 \sim SRL_3$ form a single scan chain from *SI* to *SO*.

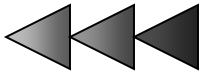
During the capture operation, clocks C_1 and C_2 are applied to load the test response from the combinational logic into the scan cells.

LSSD Design Rules



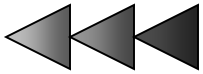
- All storage elements must be polarity-hold latches.
- The latches are controlled by two or more non-overlapping clocks.
- A set of clock primary inputs must follow three conditions:
  - All clock inputs to SRLs must be inactive when clock PIs are inactive
  - The clock input to any SRL must be controlled from one or more clock primary inputs
  - No clock can be ANDed with another clock or its complement

LSSD Design Rules



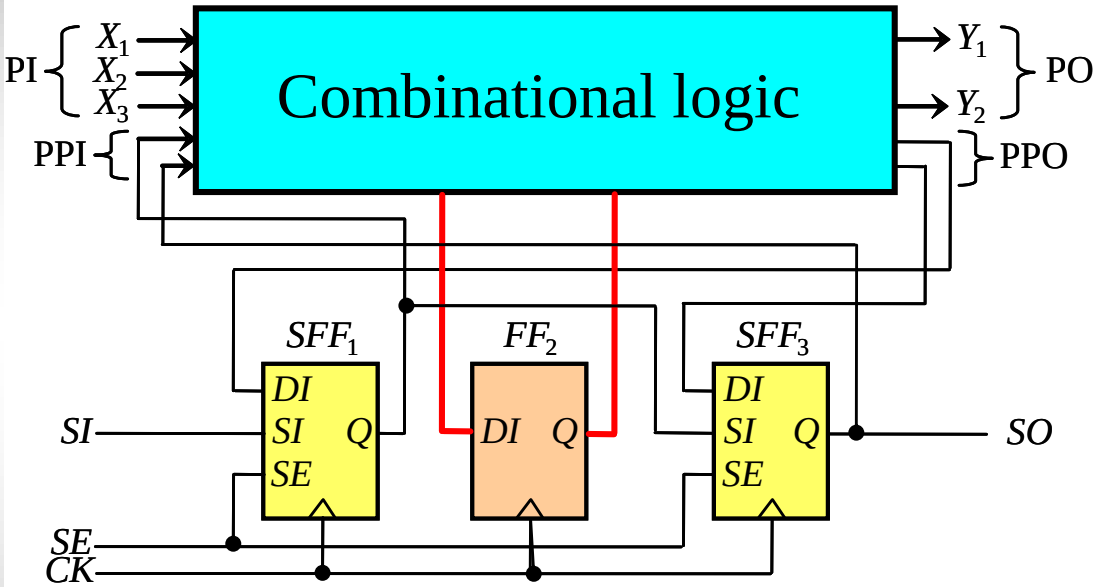
- Clock primary inputs must not feed the data inputs to SRLs either directly or through combinational logic.
- Each system latch must be part of an SRL, and each SRL must be part of a scan chain.
- A scan state exists under certain conditions:
  - Each SRL or scan out *SO* is a function of only the preceding SRL or scan input *SI* in its scan chain during the scan operation
  - All clocks except the shift clocks are disabled at the SRL clock inputs

Partial-Scan Design



- Was once used in the industry long before full-scan design became the dominant scan architecture.
- Can also be implemented using muxed-D scan cells, clocked-scan cells, or LSSD scan cells.
- Either combinational ATPG or sequential ATPG can be used.

Partial-Scan Design

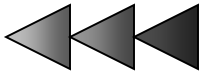


An example of muxed-D partial-scan design

A scan chain is constructed with two scan cells SFF_1 and SFF_3, while flip-flop FF_2 is left out.

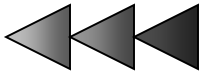
It is possible to reduce the test generation complexity by splitting the single clock into two separate clocks, one for controlling all scan cells, the other for controlling all non-scan storage elements. However, this may result in additional complexity of routing two separate clock trees during physical implementation.

Partial-Scan Design



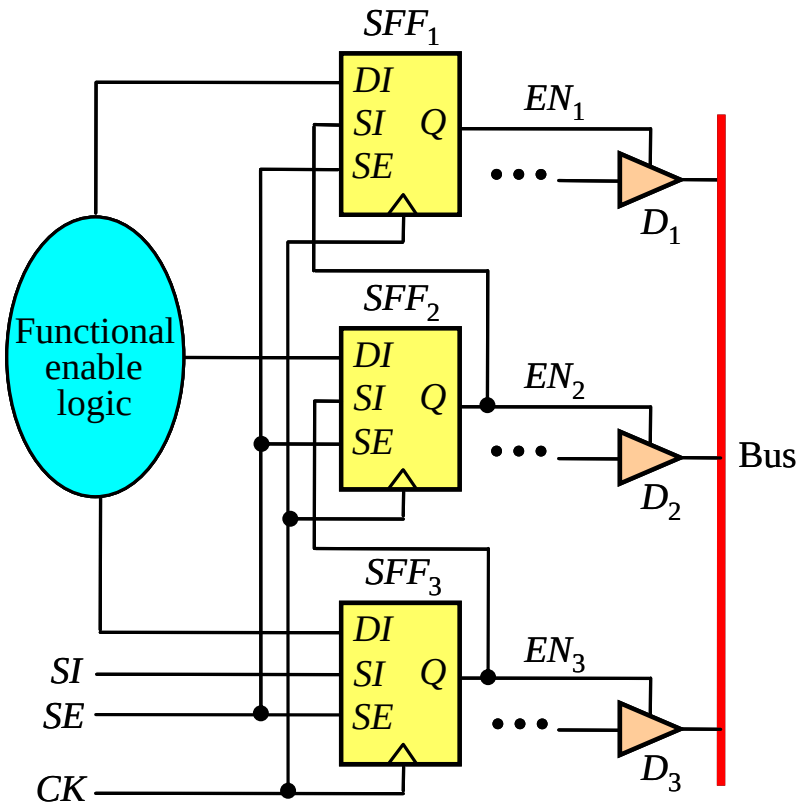
- Advantage:
  - Reduce silicon area overhead
  - Reduce performance degradation
- Disadvantage:
  - Can result in lower fault coverage
  - Longer test generation time
  - Offers less support for debug, diagnosis and failure analysis

Scan Design Rules



Design Style	*Scan Design Rule*	*Recommended Solution*
Tri-state buses	Avoid during shift	Fix bus contention during shift
Bi-directional I/O ports	Avoid during shift	Force to input or output mode during shift
Gated clocks (muxed-D full-scan)	Avoid during shift	Enable clocks during shift
Derived clocks (muxed-D full-scan)	Avoid	Bypass clocks
Combinational feedback loops	Avoid	Break the loops
Asynchronous set/reset signals	Avoid	Use external pin(s)
Clocks driving data	Avoid	Block clocks to the data portion
Floating buses	Avoid	Add bus keepers
Floating inputs	Not recommended	Tie to Vcc or ground
Cross-coupled NAND/NOR gates	Not recommended	Use standard cells
Non-scan storage elements	Not recommended for full-scan Design	Initialize to known states, bypass, or make transparent

Tri-State Buses



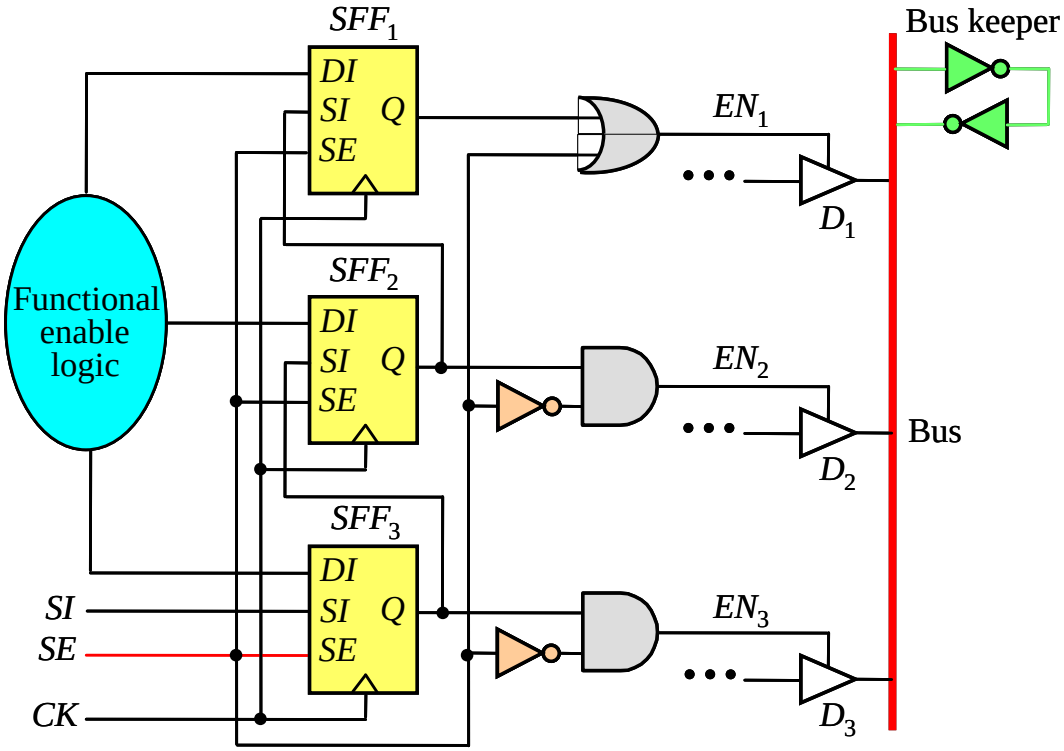
Original Circuit

Bus contention occurs when two bus drivers force opposite logic values onto a tri-state bus.

Bus contention is designed not to happen during the normal operation, and is typically avoided during the capture operation.

However, during the shift operation, no such guarantees can be made.

Tri-State Buses

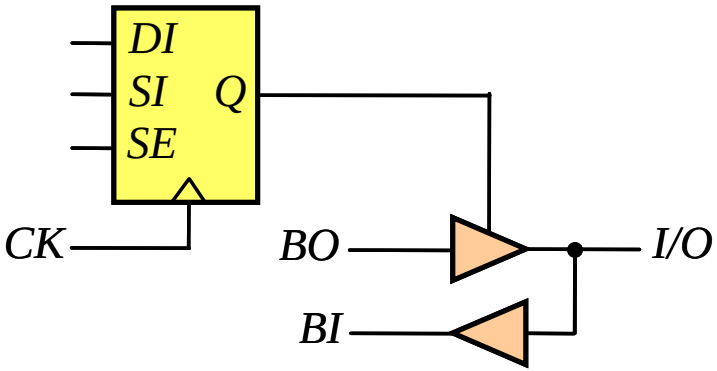


Modified circuit fixing bus contention

EN_1 is forced to 1 to enable the D_1 bus driver, while EN_2 and EN_3 are set to 0 to disable both D_2 and D_3 bus drivers, when SE = 1.

A bus without a pull-up, pull-down, or bus keeper may result in fault coverage loss, the bus keeper is added.

Bi-Directional I/O Ports

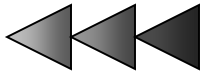


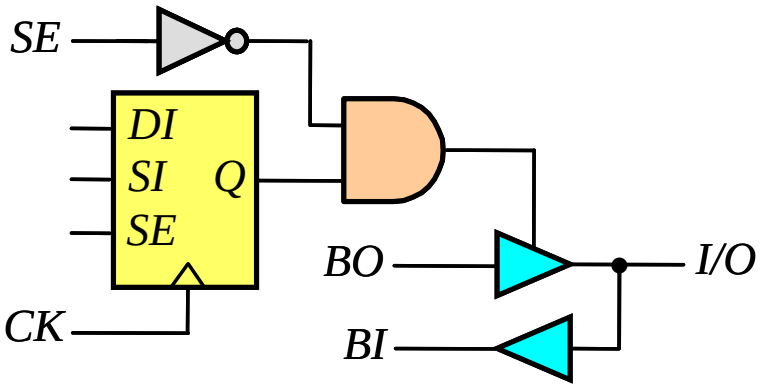
Original circuit

Conflicts may occur at a bidirectional I/O port during the shift operation.

Since the output value of the scan cell can vary during the shift operation, the output tri-state buffer may become active, resulting in a conflict if BO and the I/O port driven by the tester have opposite logic values.

Bi-Directional I/O Ports



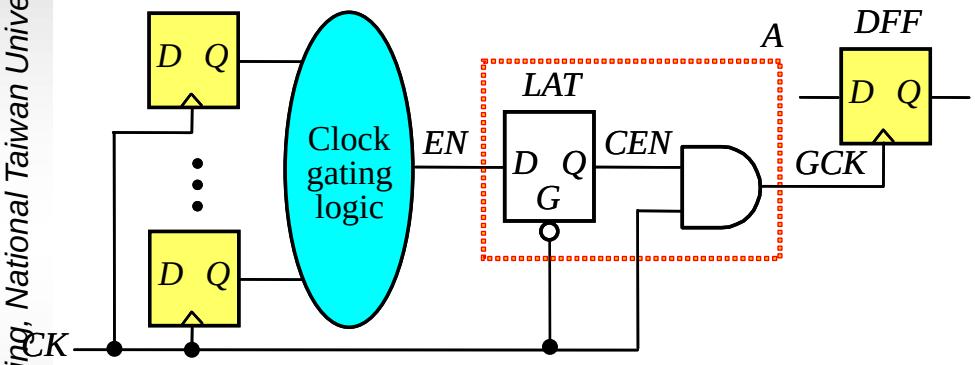


Modified circuit

Fix this problem by forcing the tri-state buffer to be inactive when *SE* = 1, and the tester is used to drive the I/O port during the shift operation.

During the capture operation, the applied test vector determines whether a bi-directional I/O port is used as input or output and controls the tester appropriately.

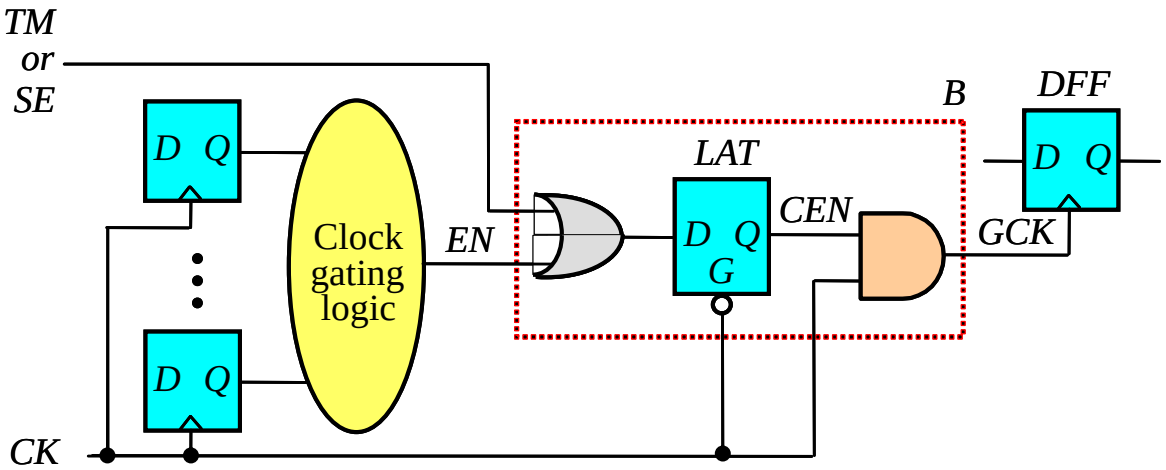
Gated Clocks



Original circuit

Although clock gating is a good approach for reducing power consumption, it prevents the clock ports of some flip-flops from being directly controlled by primary inputs.

Gated Clocks

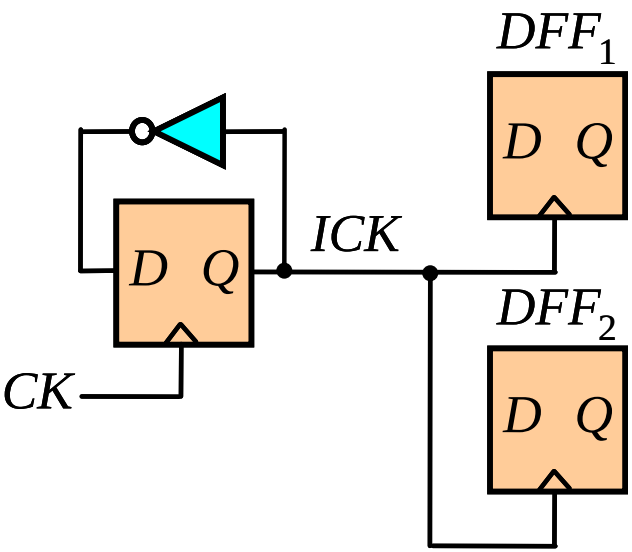


Modified Circuit

The clock gating function should be disabled at least during the shift operation.

An OR gate is used to force *CEN* to 1 using either the test mode signal *TM* or the scan enable signal *SE*.

Derived clocks

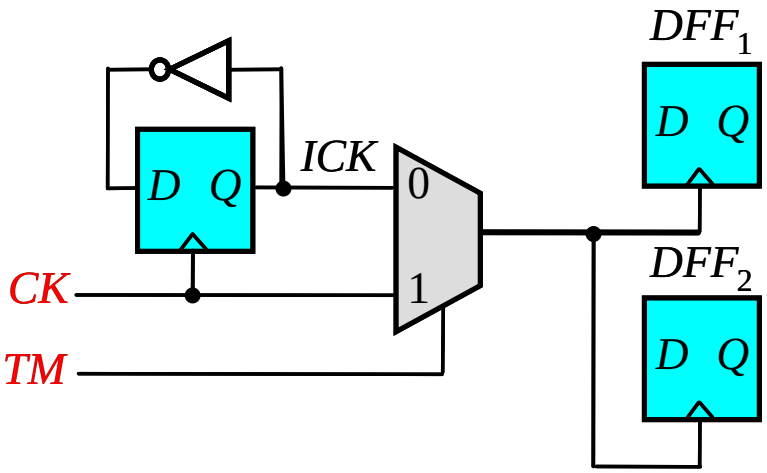


(a) Original circuit

A derived clock is a clock signal generated internally from a storage element or a clock generator.

These clock signals need to be bypassed during the entire test operation.

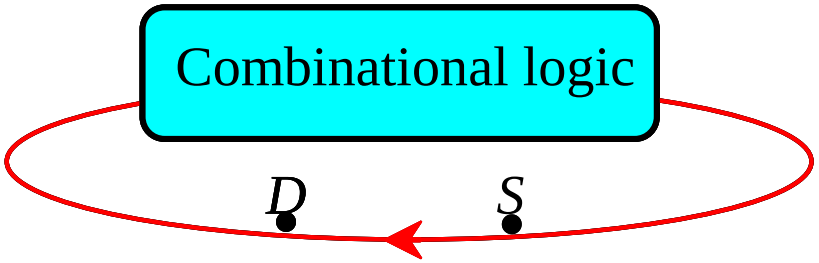
Derived clocks



(b) Modified circuit

A multiplexer selects *CK*, which is a clock directly controllable from a primary input, to drive DFF_1 and DFF_2, during the entire test operation, when $TM = 1$.

Combinational Feedback Loops



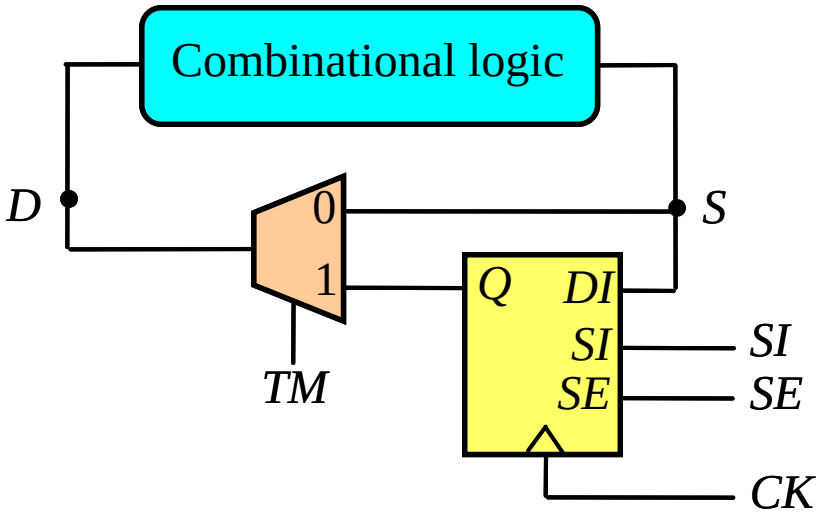
(a) Original circuit

The best way is to rewrite the RTL code.

Depending on whether the number of inversions on a combinational feedback loop is even or odd, it can introduce either sequential behavior or oscillation into a design.

Since the value stored in the loop cannot be controlled or determined during test, this can lead to an increase in test generation complexity or fault coverage loss.

Combinational Feedback Loops

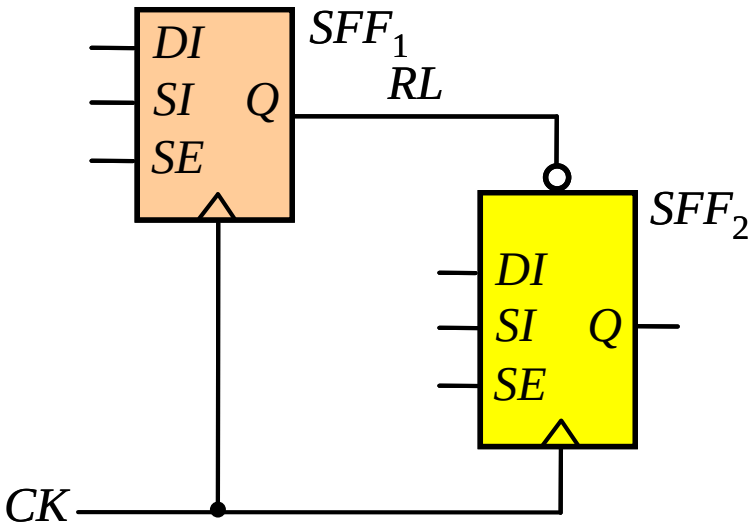


(b) Modified circuit

It can be fixed by using a test mode signal *TM*.

This signal permanently disables the loop throughout the entire shift and capture operations, by inserting a scan point to break the loop.

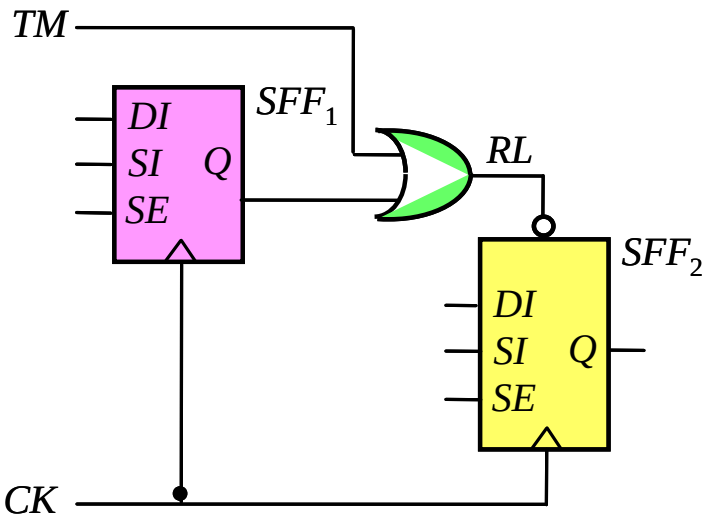
Asynchronous Set/Reset Signals



(a) Original circuit

Asynchronous set/reset signals of scan cells that are not directly controlled from primary inputs can prevent scan chains from shifting data properly.

Asynchronous Set/Reset Signals

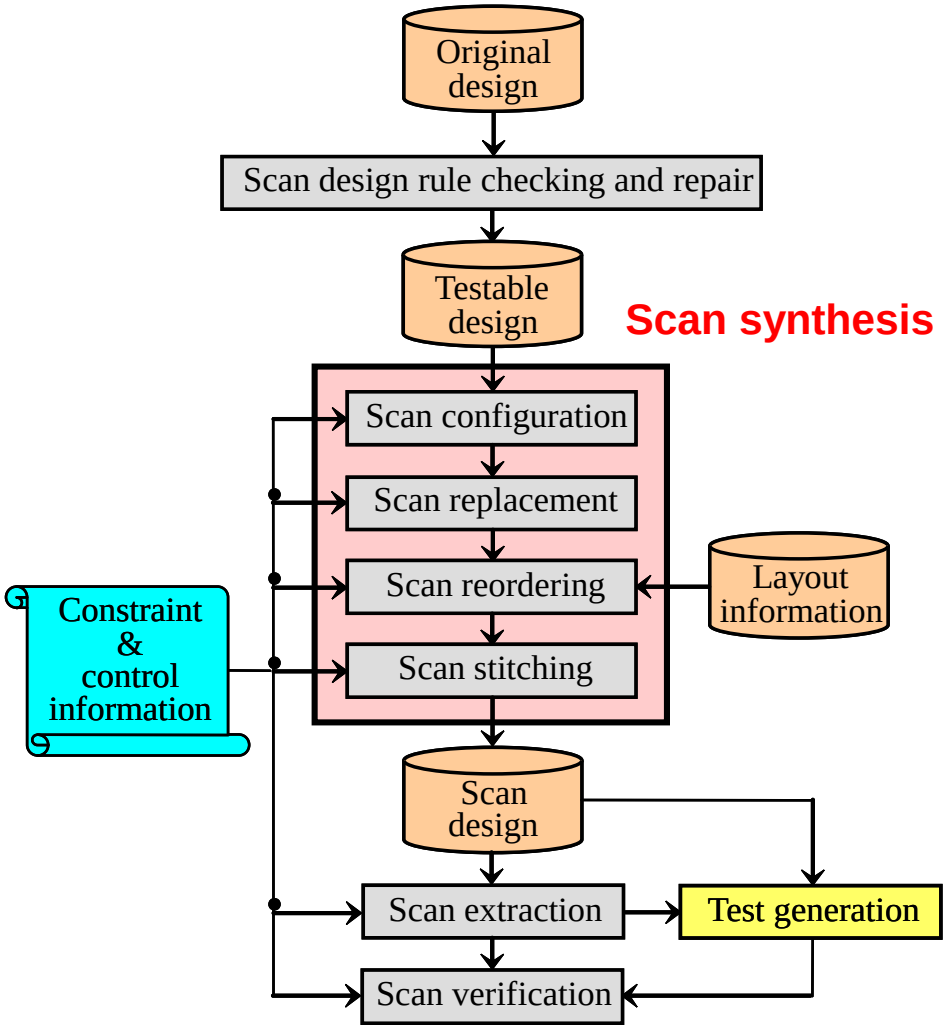


(b) Modified circuit

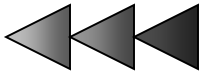
To avoid this problem, these asynchronous set/reset signals are forced to an inactive state during the shift operation.

Use an OR gate with an input tied to the test mode signal *TM*. When *TM* = 1, the asynchronous reset signal *RL* of scan cell SFF_2 is permanently disabled during the entire test operation.

Scan Design Flow

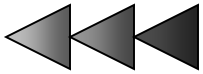


Scan Design Flow



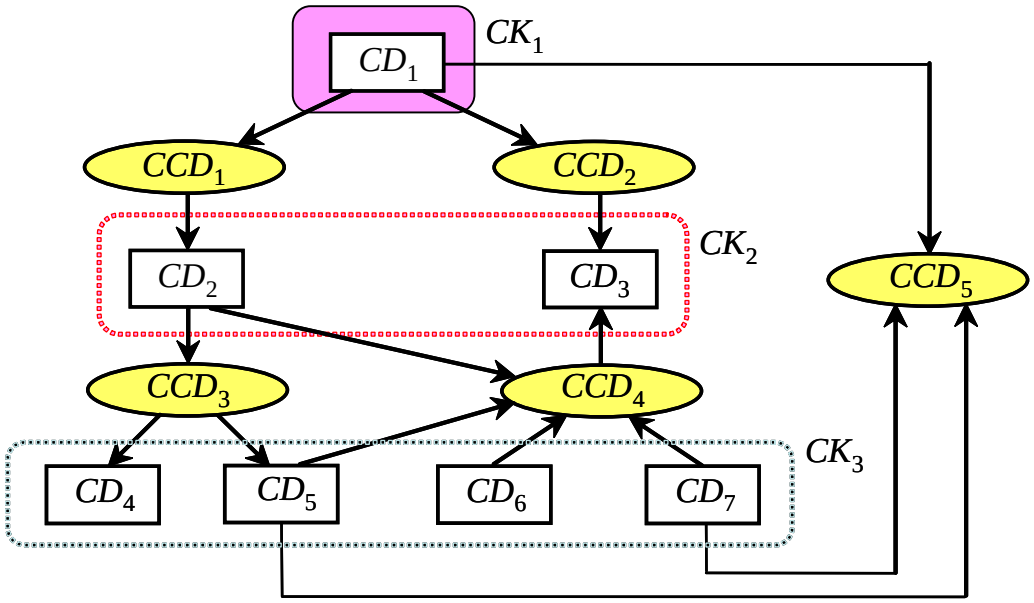
- Scan Design Rule Checking and Repair
  - Identify and repair all scan design rule violations to convert the original design into a testable design
  - Also performed after scan synthesis to confirm that no new violations exist
- Scan Synthesis
  - Converts a testable design into a scan design without affecting the functionality of the original design
    - Scan Configuration
    - Scan Replacement
    - Scan Reordering
    - Scan Stitching

Scan Design Flow



- Scan Extraction
  - Is the process used for extracting all scan cell instances from all scan chains specified in the scan design
- Scan Verification
  - A timing file in *standard delay format* (SDF) which resembles the timing behavior of the manufactured device is used to
    - Verifying the scan shift operation
    - Verifying the scan capture operation
- Scan Design Costs
  - Area overhead cost:
  - I/O pin cost
  - Performance degradation cost
  - Design effort cost

Scan Design Rule Checking and Repair



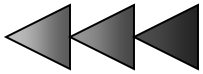
Clock grouping example

An arrow means a data transfers from one clock domain to a different clock domain.

7 clock domains, CD_1 ~ CD_7

5 crossing-clock-domain data paths, CCD_1 ~ CCD_5

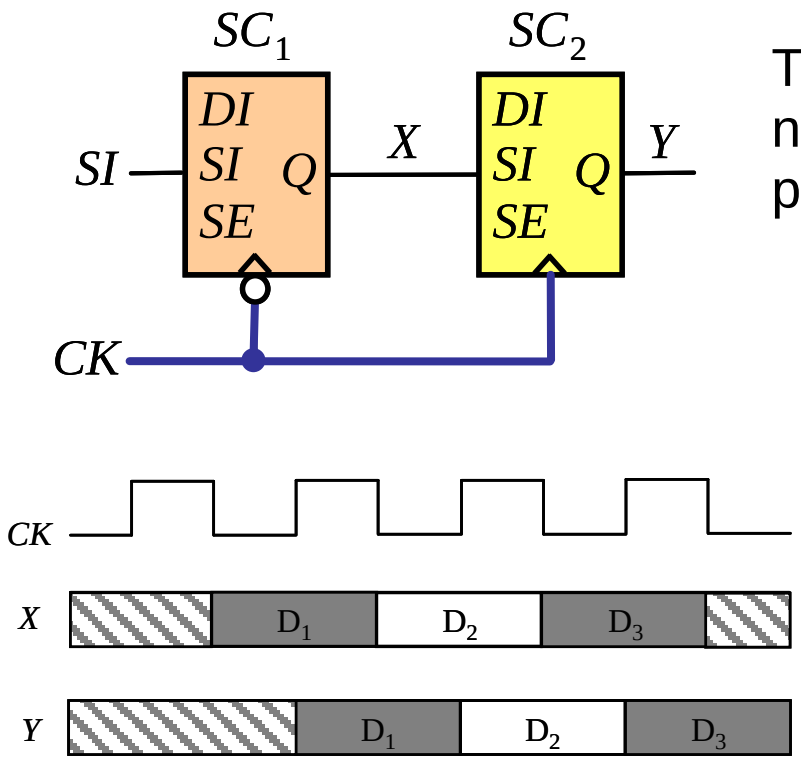
Scan Synthesis



- Includes four separate and distinct steps:
  - Scan Configuration
    - The number of scan chains used
    - The types of scan cells used to implement these scan chains
    - Which storage elements to exclude from the process
    - How the scan cells are arranged
  - Scan Replacement
    - Replaces all original storage elements in the testable design with their functionally-equivalent scan cells
  - Scan Reordering
    - The process of reordering the scan chains based on the physical scan cell locations, in order to minimize the amount of interconnect wires used to implement the scan chains
  - Scan Stitching
    - Stitch all scan cells together to form scan chains

Scan Synthesis - Scan Configuration

Mixing negative-edge and positive-edge scan cells in a scan chain

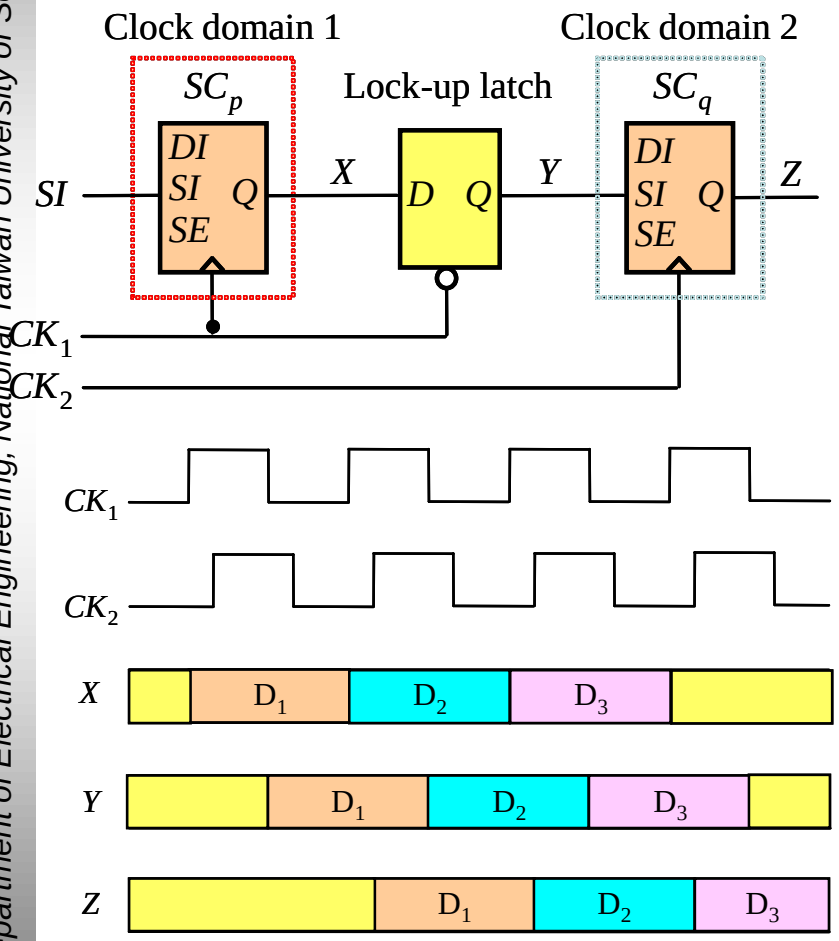


This circuit structure comprising a negative-edge scan cell followed by a positive-edge scan cell.

Y will first take on the state *X* at the rising *CK* edge, before *X* is loaded with the *SI* value at the falling *CK* edge.

If we accidentally place the positive-edge scan cell before the negative-edge scan cell, both scan cells will always incorrectly contain the same value at the end of each shift clock cycle.

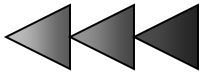
Scan Synthesis-Scan Configuration



A lock-up latch is inserted between adjacent cross-clock-domain scan cells, in order to guarantee that any clock skew between the clocks can be tolerated.

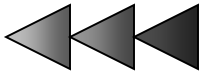
During each shift clock cycle, *X* will first take on the *SI* value at the rising CK_1 edge. Then, *Z* will take on the *Y* value at the rising CK_2 edge.

Snapshot scan



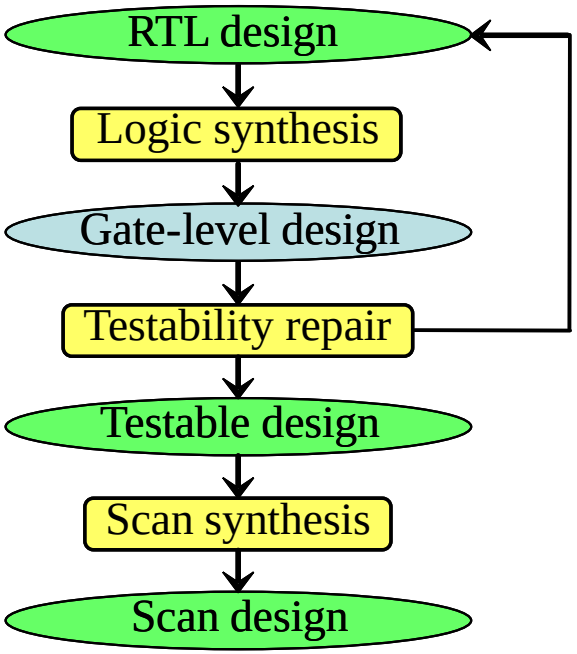
- Why snapshot scan is introduced?
  - Capture a snapshot of the internal states
  - Without disruption of the functional operation
- What is new?
  - Add a scan cell (2-port D latches) to each storage element of interest
  - Implement scan-set architecture

RTL Design for Testability

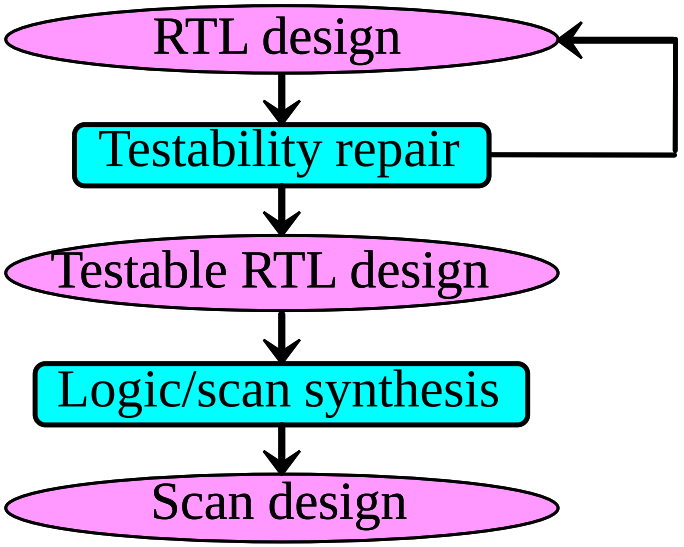


- Why are RTL designs needed?
  - Growth of device number
  - Tight timing
  - Potential yield loss
  - Low-power issues
  - Increased core reusability
  - Time-to market pressure

Comparison of design flows at RTL and Gate-level

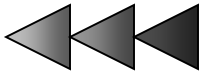


Gate-level testability repair design flow



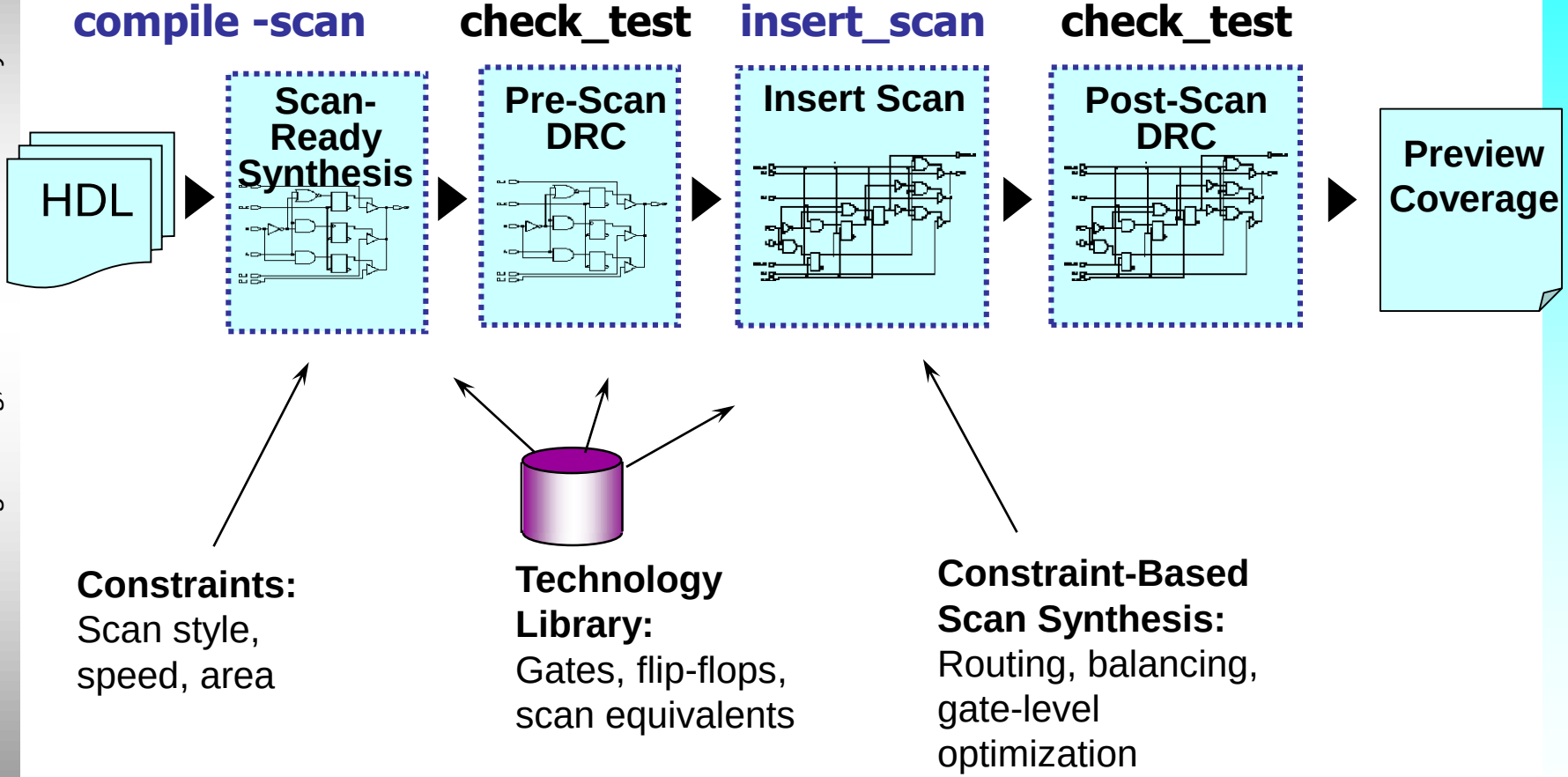
RTL testability repair design flow

RTL Scan Design Rule Checking



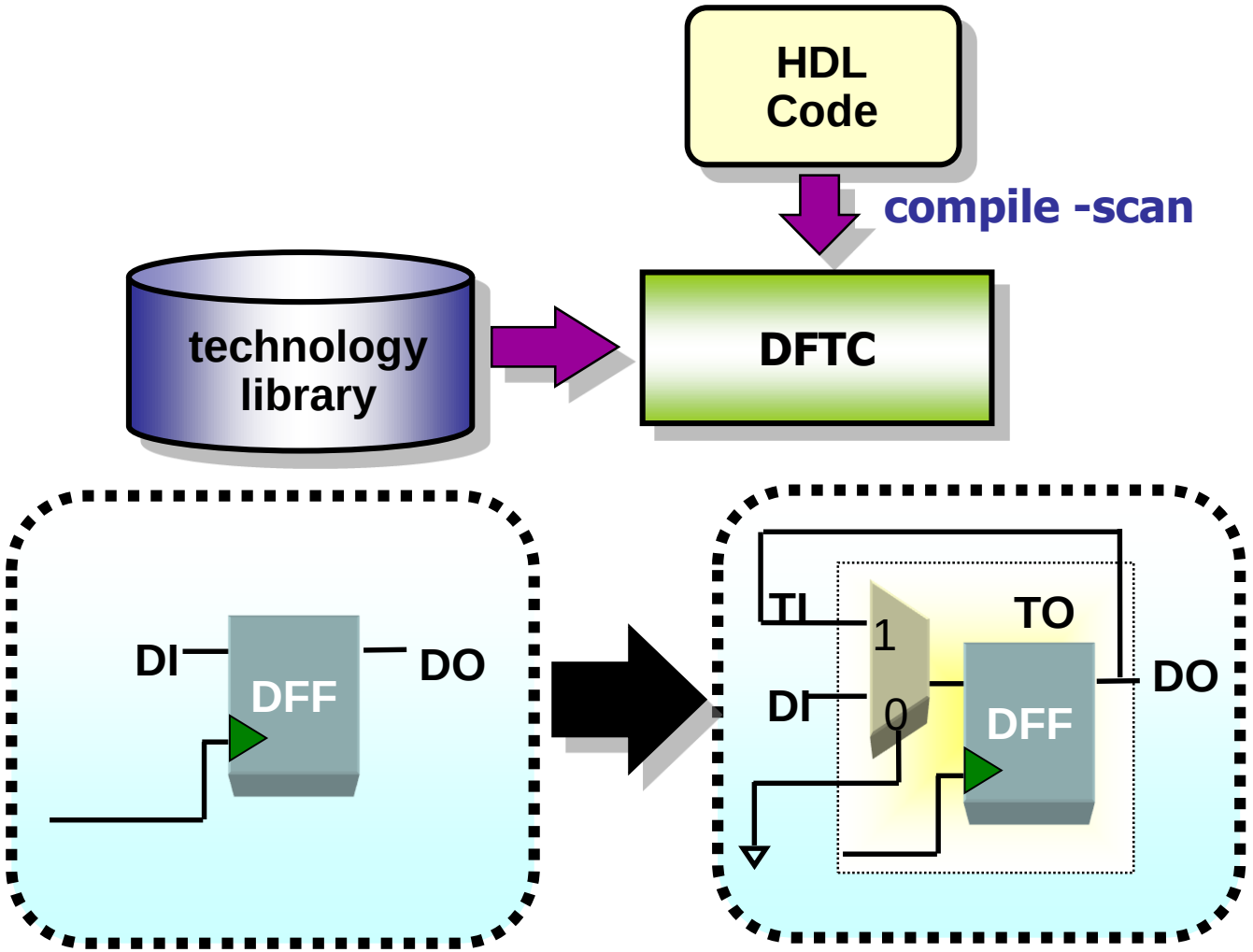
- Fast synthesis
  - Mapped onto combinational primitives and high-level models
- Identify testability problems
  - Static solutions (without simulation)
  - Dynamic solutions (with simulation)

Synopsys DFT Compiler Flow

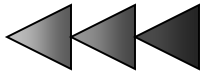


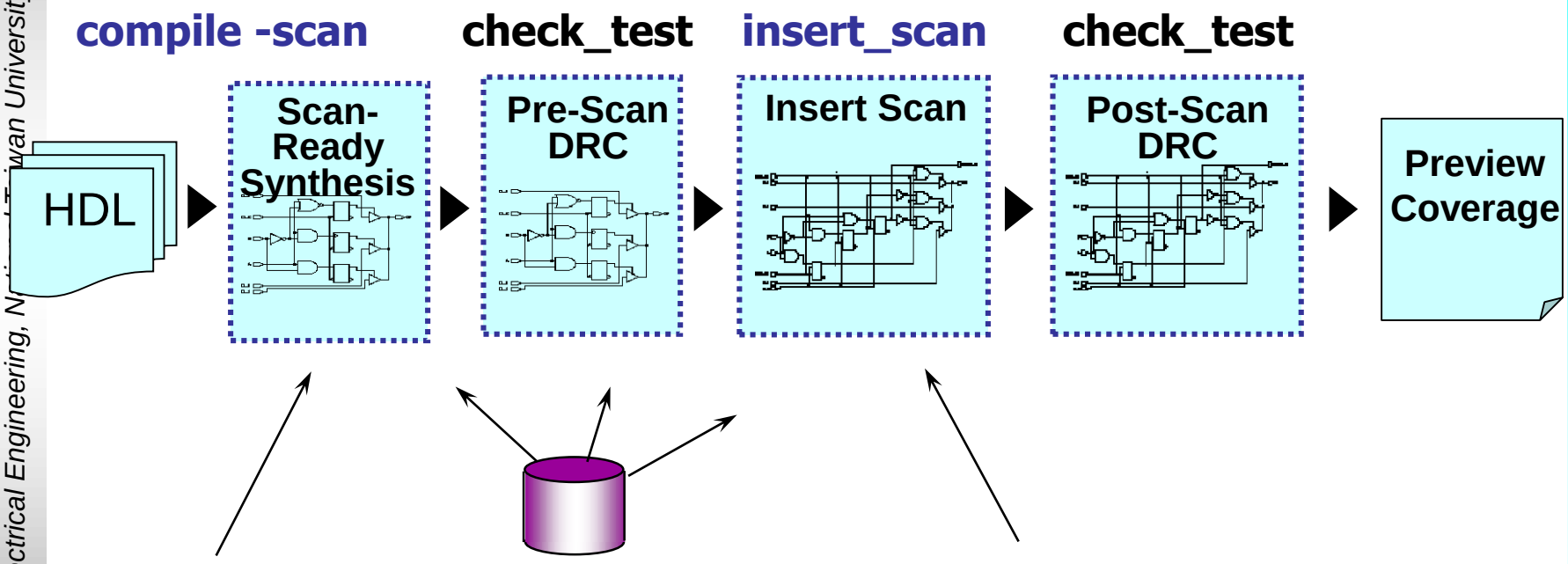
Source: CIC

Scan-Ready Synthesis



Synopsys DFT Compiler Flow



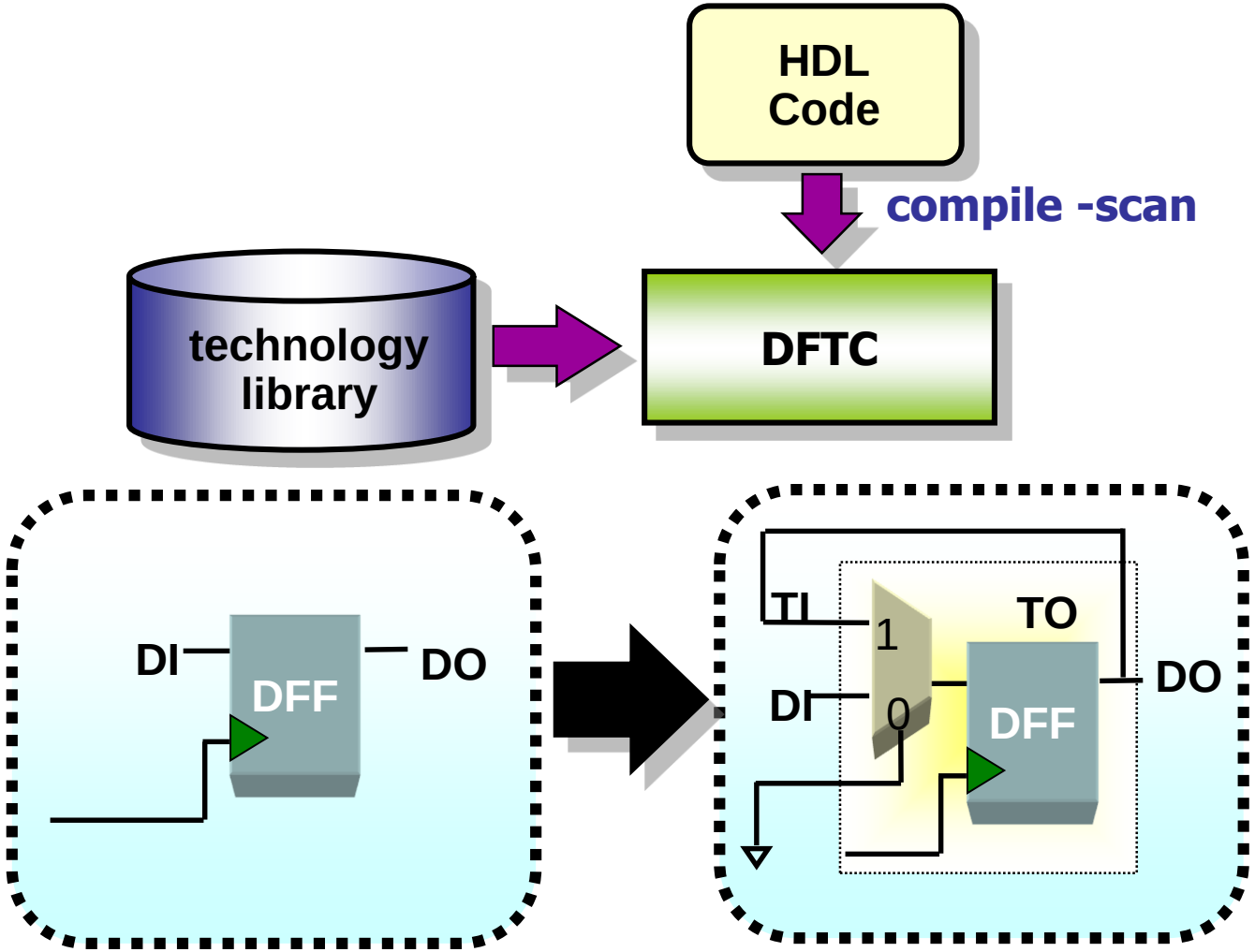


Constraints:
Scan style, speed, area

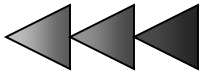
Technology Library:
Gates, flip-flops, scan equivalents

Constraint-Based Scan Synthesis:
Routing, balancing, gate-level optimization

Scan-Ready Synthesis



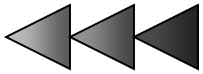
Pre-Scan Check



- Check gate-level scan design rule **before** scan chain synthesis.
- Looks at four categories of testability issues:
  - Modeling problems, such as lack of a scan equivalent.
  - Topological problems, like unclocked feedback loops.
  - Protocol inference, such as test clocks and test holds.
  - Protocol simulation, to verify proper scanning of bits.

```
dc_shell> check_test
...basic checks...
...checking combinational feedback loops...
...inferring test protocol...
Inferred system/test clock port CLK (45.0,55.0).
...simulating parallel vector...simulating serial scan-in…
Information: The first scan-in cycle does not shift in data.(TEST-301)
Warning: Cell U1 (FD1S) is not scan controllable. (TEST-302)
Information: Because it clocks in an unknown value from pin TI.(TEST-512)
Information: Because port SI is unknown. (TEST-514)
Information: As a result, 3 other cells are not scan controllable.(TEST-502)
Information: Test design rule checking completed. (TEST-123)
```

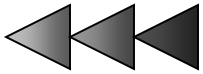
Scan Chain Insertion



- Scan-Chain Insertion Algorithm:

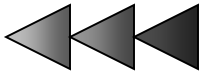
1. Targets the previewed scan-path architecture.
2. Performs any remaining scan replacements.
3. Adds disabling/enabling logic to tristate buses.
4. Conditions the directionality of bidirectional ports.
5. Wires the scan flops into the specified chains.
6. Optimizes the logic, minimizing constraint violations.

Post-Scan Check



- Why run check_test again?
  - Confirm there are no new DFT problems.
  - Verify the scan chains synthesized operates properly.
  - Create an ATPG-ready database.

Estimate Test coverage

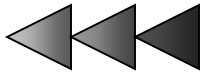


- Use the DFTC ATPG command:
  estimate_test_coverage will call TetraMAX for fault estimate.

```
                    Pattern Summary Report

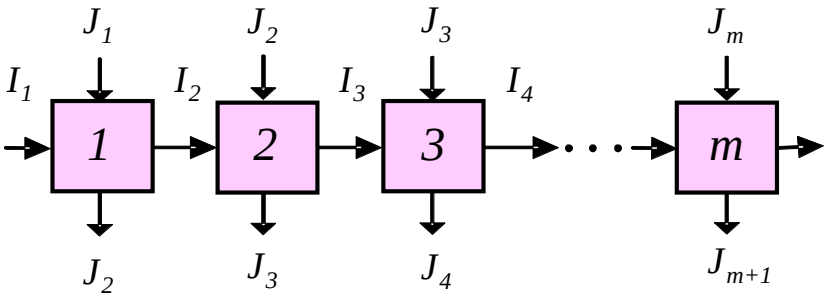
          Uncollapsed Stuck Fault Summary Report
    ---------------------------------------------
    fault class                      code   #faults
    -------------------------------  ----  ---------
    Detected                          DT       3084
    Possibly detected                 PT          0
    Undetectable                      UD         12
    ATPG untestable                   AU          0
    Not detected                      ND          0
    ---------------------------------------------
    total faults                                3096
    test coverage                             100.00%
    ---------------------------------------------
       Information: The test coverage above may be
    Inferior than the real test coverage with customized
              protocol and test simulation library.
```

Concluding Remarks

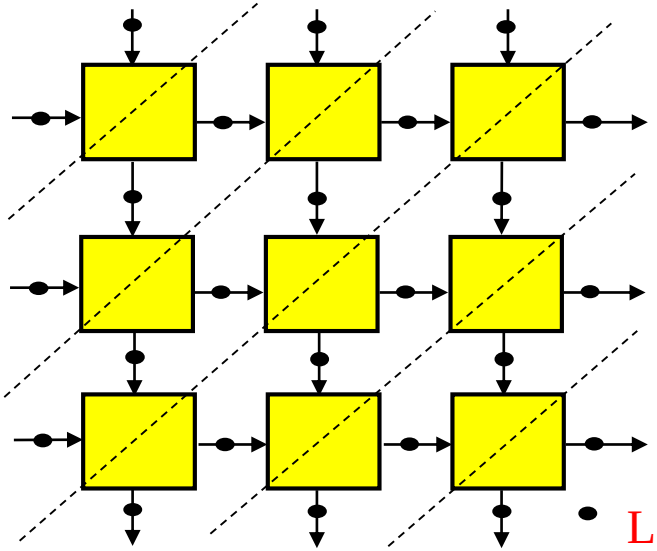


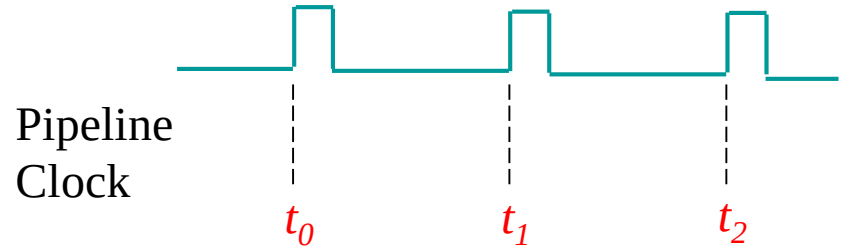
- DFT has become vital for ensuring product quality
- Scan design is the most widely used DFT technique
- New design and test challenges
  - Further reduce test power, test data volume and test application time
  - Cope with physical failures of the nanometer design era

C-Testable Design Techniques for CMOS Iterative Logic Arrays



1-D Iterative Logic Arrays



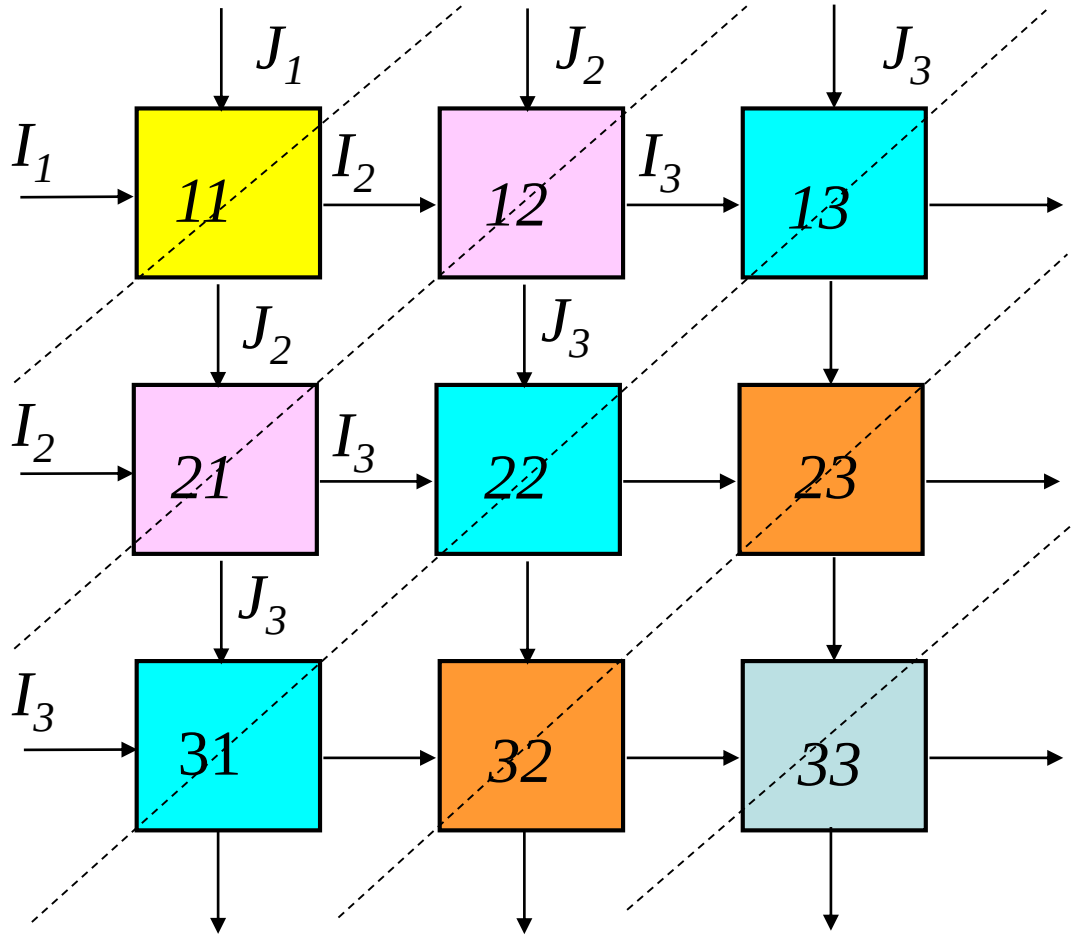


2-D Iterative Logic Arrays

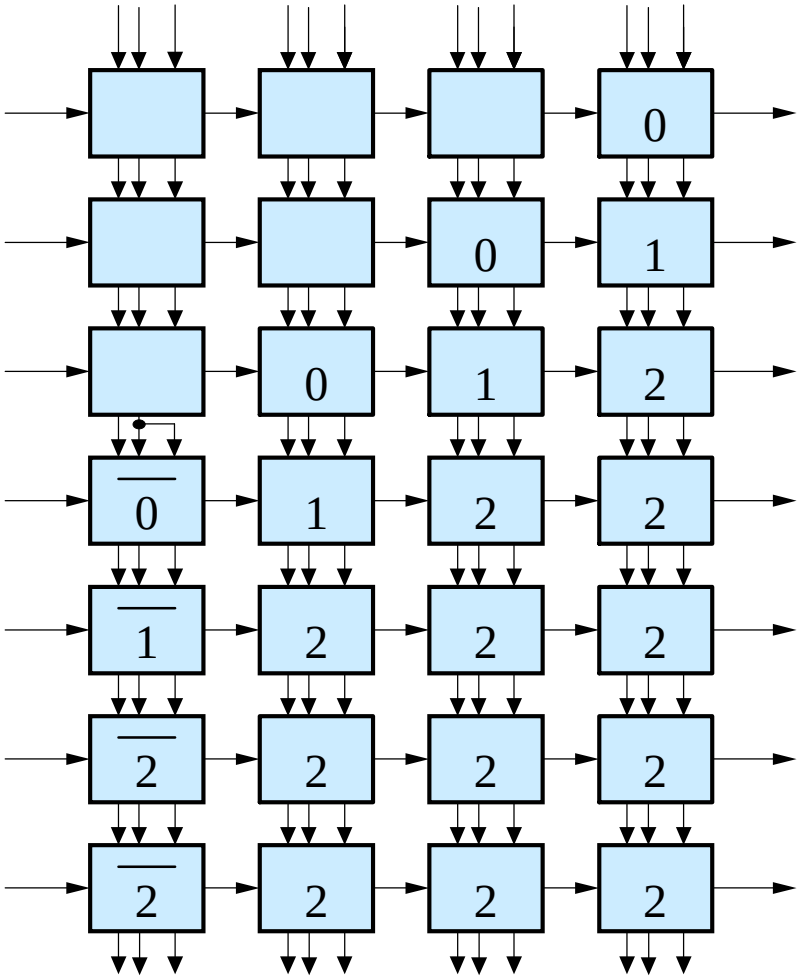
Definitions

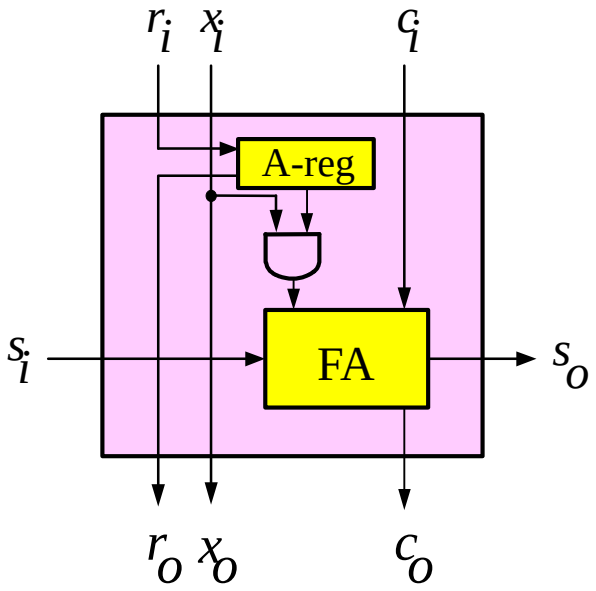
- A cell a combinational machine (Σ,Δ,f), where $f : \Sigma \rightarrow \Delta$, $\Sigma = \{0, 1\}^I$,and$\Delta = \{0, 1\}^O$ for $I, O \in N$. An ILA is an array of cells. Unless otherwise specified, cells are assumed to be functionally identical, which implies that $\Sigma=\Delta$, so $I = O \equiv k$
- We say that an array is M-testable when it is 2^k-testable when it is 2^k-testable, i.e., testable with 2^k patterns.
- A minimal complete sequence is a shortest such sequence. We say a cell function f is injective if $\forall(i_1 ,j_1) \neq (i_2 ,j_2)$, $f(i_1 ,j_1) \neq f(i_2 ,j_2)$. We say f is bijective if it is injective and $\Sigma=\Delta$.

Test Tessellaton of a mesh-connected iterative logic array



Bit-level array implementation of an 4 × 3 array multiplier





$$x_o = x_i$$

$$r_o = r_i$$

$$s_o = s_i \oplus c_i \oplus r_i x_i$$

$$c_o = s_i c_i + c_i r_i x_i + s_i r_i x_i$$

Truth tables of the original multiplier cell (a) A = 0; (b) A = 1.

x_i	s_i	c_i	x_o	s_o	c_o
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	0	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

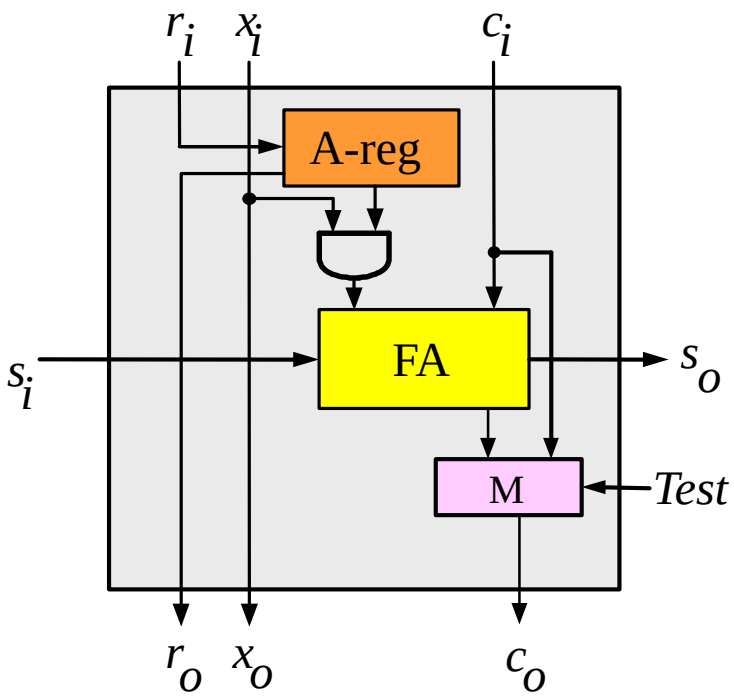
x_i	s_i	c_i	x_o	s_o	x_o
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	0	1
1	1	0	1	0	1
1	1	1	1	1	1

Truth tables of the modified multiplier cell (a) A = 0; (b) A = 1

x_i	s_i	c_i	x_o	s_o	c_o
0	0	**0**	0	0	**0**
0	0	**1**	0	1	**1**
0	1	**0**	0	1	**0**
0	1	**1**	0	0	**1**
1	0	**0**	1	0	**0**
1	0	**1**	1	1	**1**
1	1	**0**	1	1	**0**
1	1	**1**	1	0	**1**

x_i	s_i	c_i	x_o	s_o	c_o
0	0	**0**	0	0	**0**
0	0	**1**	0	1	**1**
0	1	**0**	0	1	**0**
0	1	**1**	0	0	**1**
1	0	**0**	1	1	**0**
1	0	**1**	1	0	**1**
1	1	**0**	1	0	**0**
1	1	**1**	1	1	**1**

The modified multiplier cell



$$x_o = x_i$$

$$r_o = r_i$$

$$s_o = s_i \oplus c_i \oplus r_i x_i$$

$$c_o = (s_i c_i + c_i r_i x_i + s_i r_i x_i)\overline{Test} + c_i Test,$$